

Multilayer Perceptrons



Understanding Multilayer Perceptrons

Artificial neural networks form a fascinating area of study, although they can be intimidating when you are just getting started. There are a lot of specialized terms in use to describe the data structures and algorithms used in the field. In this chapter, you will get a crash course in the terminology and processes used in the field of multilayer perceptron artificial neural networks. After completing this chapter, you will know:

- ▷ The building blocks of neural networks, including neurons, weights, and activation functions
- ▷ How the building blocks are used in layers to create networks
- ▷ How networks are trained from example data

Let's get started.

Overview

This chapter is in four parts; they are:

- ▷ Multilayer Perceptrons
- ▷ Neurons: Weights and Activations
- ▷ Networks of Neurons
- ▷ Training Networks

5.1 Multilayer Perceptrons

The field of artificial neural networks is often just called *neural networks* or *multilayer perceptrons*, as multilayer perceptron is perhaps the most useful type of neural network. A perceptron is a single neuron model that was a precursor to larger neural networks. It is a field of study that investigates how simple models of biological brains can be used to solve difficult computational tasks like the predictive modeling tasks we see in machine learning. The goal is not to create realistic models of the brain but instead to develop robust algorithms and data structures that we can use to model difficult problems.

The power of neural networks come from their ability to learn the correlations in your training data and how best to relate it to the output variable you want to predict. In this sense, neural networks learn mapping. Mathematically, they are capable of learning any mapping function and have been proven to be a universal approximation algorithm. The predictive capability of neural networks comes from the hierarchical or multilayered structure of the networks. The data structure can pick out (learn to represent) features at different scales or resolutions and combine them into higher-order features, for example, from lines to collections of lines to shapes.

5.2 Neurons: Weights and Activations

The building blocks for neural networks are artificial neurons. These are simple computational units that have weighted input signals and produce an output signal using an activation function.

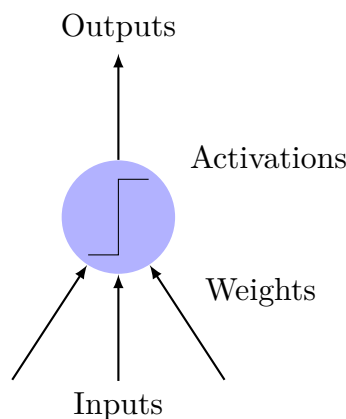


Figure 5.1: Model of a simple neuron

Neuron Weights

You may be familiar with linear regression, where the weights on the inputs are very much like the coefficients used in a regression equation. Like linear regression, each neuron also has a bias which can be thought of as an input that always has the value 1.0, and it, too, must be weighted. For example, a neuron may have two inputs, which requires three weights — one for each input and one for the bias.

Weights are often initialized to small random values, such as values from 0 to 0.3, although more complex initialization schemes can be used. Like linear regression, larger weights indicate increased complexity and fragility of the model. Keeping weights in the network small is desirable, and regularization techniques can be used.

Activation

The weighted inputs are summed and passed through an activation function, sometimes called a transfer function. An activation function is a simple mapping of summed weighted input to

the output of the neuron. It is called an activation function because it governs the threshold at which the neuron is activated and the strength of the output signal. Historically, simple step activation functions were used. When the summed input was above a threshold of 0.5, for example, then the neuron would output a value of 1.0; otherwise it would output a 0.0.

Traditionally, nonlinear activation functions are used. This allows the network to combine the inputs in more complex ways and, in turn, provide a richer capability in the functions they can model. Nonlinear functions like the logistic function, also called the sigmoid function, were used to output a value between 0 and 1 with an S-shaped distribution. The hyperbolic tangent function, also called tanh, were used to output the same distribution over the range -1 to $+1$. More recently, the rectifier activation function has been shown to provide better results.

5.3 Networks of Neurons

Neurons are arranged into networks of neurons, the figure below is an example. A row of neurons is called a layer, and one network can have multiple layers. The architecture of the neurons in the network is often called the network topology.

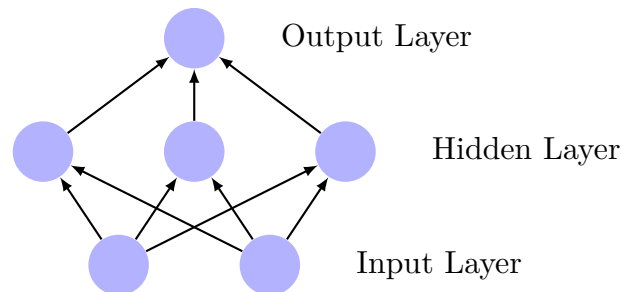


Figure 5.2: Model of a simple network

Input or Visible Layers

The bottom layer that takes input from your dataset is called the visible layer because it is the exposed part of the network. Often a neural network is drawn with a visible layer with one neuron per input value or column in your dataset. These are not actual neurons as described above, but simply drawn as nodes to represent input values that will be passed to the next layer.

Hidden Layers

Layers after the input layer are called hidden layers because they are not directly exposed to the input. The simplest network structure is to have a single neuron in the hidden layer that directly outputs the value. Given increases in computing power and efficient libraries, very deep neural networks can be constructed. Deep learning can refer to having many hidden layers in your neural network. They are deep because they would have been unimaginably slow to train historically but may take seconds or minutes to train using modern techniques and hardware.

Output Layer

The final hidden layer is called the output layer, and it is responsible for outputting a value or vector of values that correspond to the format required for the problem. The choice of activation function in the output layer is strongly constrained by the type of problem that you are modeling. For example:

- ▷ A regression problem may have a single output neuron, and the neuron may have no activation function.
- ▷ A binary classification problem may have a single output neuron and use a sigmoid activation function to output a value between 0 and 1 to represent the probability of the primary class as predicted. This can be turned into a crisp class value by using a threshold of 0.5 and snap values less than the threshold to 0, otherwise to 1.
- ▷ A multiclass classification problem may have multiple neurons in the output layer, one for each class (e.g., three neurons for the three classes in the famous iris flowers classification problem). In this case, a softmax activation function may be used to output a probability of the network predicting each of the class values. Selecting the output with the highest probability can be used to produce a crisp classification value.

5.4 Training Networks

Once configured, the neural network needs to be trained on your dataset.

Data Preparation

You must first prepare your data for training on a neural network. Data must be numerical, for example, real values. If you have categorical data, such as a *sex* attribute with the values “male” and “female”, you can convert it to a real-valued representation called one-hot encoding. This is where one new column is added for each class value (two columns in the case of sex of male and female), and a 0 or 1 is added for each row depending on the class value for that row.

This same one-hot encoding can be used on the output variable in classification problems with more than one class. This would create a binary vector from a single column that would be easy to directly compare to the output of the neuron in the network’s output layer. That, as described above, would output one value for each class. Neural networks require the input to be scaled in a consistent way. You can rescale it to the range between 0 and 1, this is called normalization. Another popular technique is to standardize it so that the distribution of each column has a mean of zero and a standard deviation of 1. Scaling also applies to image pixel data. Data such as words can be converted to integers, such as the frequency rank of the word in the dataset and other encoding techniques.

Stochastic Gradient Descent

The classical and still preferred training algorithm for neural networks is called stochastic gradient descent. This is where one row of data is exposed to the network at a time as input. The network processes the input upward, activating neurons as it goes to finally produce an

output value. This is called a forward pass on the network. It is the type of pass that is also used after the network is trained in order to make predictions on new data.

The output of the network is compared to the expected output, and an error is calculated. This error is then propagated back through the network, one layer at a time, and the weights are updated according to the amount they contributed to the error. This clever bit of math is called the backpropagation algorithm. The process is repeated for all of the examples in your training data. One round of updating the network for the entire training dataset is called an epoch. A network may be trained for tens, hundreds, or many thousands of epochs.

Weight Updates

The weights in the network can be updated from the errors calculated for each training example, and this is called *online learning*. It can result in fast but also chaotic changes to the network. Alternatively, the errors can be saved across all the training examples, and the network can be updated at the end. This is called *batch learning* and is often more stable.

Because datasets are so large and because of computational efficiencies, the size of the batch, the number of examples the network is shown before an update, is often reduced to a small number, such as tens or hundreds of examples. The amount that weights are updated is controlled by a configuration parameter called the *learning rate*. It is also called the *step size* and controls the step or change made to network weights for a given error. Often small learning rates are used, such as 0.1 or 0.01 or smaller. The update equation can be complemented with additional configuration terms that you can set.

- ▷ **Momentum** is a term that carry over the weight update of the previous step to the current, so the weight update will continue in the same direction even when there is less error being calculated.
- ▷ **Learning Rate Decay** is used to decrease the learning rate over epochs to allow the network to make large changes to the weights at the beginning and smaller fine-tuning changes later in the training schedule.

Prediction

Once a neural network has been trained, it can be used to make predictions. You can make predictions on test or validation data in order to estimate the skill of the model on unseen data. You can also deploy it operationally and use it to make predictions continuously. The network topology and the final set of weights are all you need to save from the model. Predictions are made by providing the input to the network and performing a forward-pass, allowing it to generate an output you can use as a prediction.

5.5 More Resources

There are decades of papers and books on the topic of artificial neural networks. If you are new to the field, check out the following resources as further reading:

Books

Christopher M. Bishop. *Neural Networks for Pattern Recognition*. Oxford University Press, 1996.

<https://www.amazon.com/dp/0198538642>

Russell D. Reed and Robert J. Marks II. *Neural Smithing: Supervised Learning in Feedforward Artificial Neural Networks*. MIT Press, 1999.

<https://www.amazon.com/dp/0262527014>

Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.

<https://www.amazon.com/dp/0262035618>

(Online version at <http://www.deeplearningbook.org>).

5.6 Summary

In this chapter, you discovered artificial neural networks for machine learning. You learned:

- ▷ How neural networks are not models of the brain but are instead computational models for solving complex machine learning problems.
- ▷ That neural networks are comprised of neurons that have weights and activation functions.
- ▷ The networks are organized into layers of neurons and are trained using stochastic gradient descent.
- ▷ That it is a good idea to prepare your data before training a neural network model.

In the next chapter you will learn the structure and the building blocks of a multilayer perceptron model in Keras.

Building Multilayer Perceptron Models in Keras

The Keras Python library for deep learning focuses on the creating models as a sequence of layers. In this chapter, you will discover the simple components you can use to create neural networks and simple deep learning models using Keras from TensorFlow.

Let's get started.

Overview

This chapter is in seven parts; they are:

- ▷ Neural Network Models in Keras
- ▷ Model Input
- ▷ Model Layers
- ▷ Model Compilation
- ▷ Model Training
- ▷ Model Prediction
- ▷ Summarize the model

6.1 Neural Network Models in Keras

The focus of the Keras library is a model. The simplest model is defined in the `Sequential` class, which is a linear stack of Layers. You can create a `Sequential` model and define all the layers in the constructor; for example:

```
from tensorflow.keras.models import Sequential
model = Sequential(...)
```

Listing 6.1: Setting up a `Sequential` model in Keras

A more useful idiom is to create a `Sequential` model and add your layers in the order of the computation you wish to perform; for example:


```
from tensorflow.keras.models import Sequential
model = Sequential()
model.add(...)
model.add(...)
model.add(...)
```

Listing 6.2: Adding layers into a `Sequential` model

6.2 Model Inputs

The first layer in your model must specify the shape of the input. This is the number of input attributes defined by the `input_shape` argument. This argument expects a tuple. For example, you can define input in terms of 8 inputs for a `Dense` type layer as follows:

```
from tensorflow.keras.layers import Dense
layer = Dense(16, input_shape=(8,))
```

Listing 6.3: Defining a fully-connected layer

6.3 Model Layers

Layers of different types have a few properties in common, specifically their method of weight initialization and activation functions.

Weight Initialization

The type of initialization used for a layer is specified in the `kernel_initializer` argument. Some common types of layer initialization include:

- ▷ `random_uniform`: Weights are initialized to small uniformly random values between -0.05 and $+0.05$.
- ▷ `random_normal`: Weights are initialized to small Gaussian random values (zero mean and standard deviation of 0.05).
- ▷ `zero`: All weights are set to zero values.

For example, you can create a layer initialized with random uniform weights:

```
from tensorflow.keras.layers import Dense
layer = Dense(16, input_shape=(8,), kernel_initializer="random_uniform")
```

Listing 6.4: Defining a fully-connected layer with random uniform weight

You can see a full list of the initialization techniques supported on the *Layer Weight Initializers* page.

Activation Function

Keras supports a range of standard neuron activation functions, such as softmax, rectified linear (ReLU), tanh, and sigmoid. You typically specify the type of activation function used by a layer in the `activation` argument, which takes a string value. You can see a full list of activation functions supported by Keras on the *Layer activation functions* page.

A layer usually has no activation by default, or effectively using a linear activation of $f(x) = x$. We need to specify an activation function if it is not desirable, as follows:

```
from tensorflow.keras.layers import Dense
layer = Dense(16, input_shape=(8,),
              kernel_initializer="random_uniform", activation="relu")
```

Listing 6.5: Defining a fully-connected layer with random uniform weight and ReLU activation

Interestingly, you can also create an `Activation` object and add it directly to your model after your layer to apply that activation to the output of the layer.

Layer Types

There are a large number of core layer types for standard neural networks. Some common and useful layer types you can choose from are:

- ▷ **Dense:** Fully connected layer and the most common type of layer used on multilayer perceptron models. This is the one you saw above
- ▷ **Dropout:** Apply dropout to the model, setting a fraction of inputs to zero in an effort to reduce overfitting
- ▷ **Concatenate:** Combine the outputs from multiple layers as input to a single layer

You can learn about the full list of core Keras layers on the *Keras Layers API* page.

6.4 Model Compilation

Once you have defined your model, it needs to be compiled. This creates the efficient structures used by TensorFlow in order to efficiently execute your model during training. Specifically, TensorFlow converts your model into a graph so the training can be carried out efficiently.

You compile your model using the `compile()` function, and it accepts three important attributes:

1. Model optimizer
2. Loss function
3. Metrics

```
model.compile(optimizer=..., loss=..., metrics=...)
```

Listing 6.6: Syntax of compiling a Keras model

Model Optimizers

The optimizer is the search technique used to update weights in your model. You can create an optimizer object and pass it to the compile function via the optimizer argument. This allows you to configure the optimization procedure with its own arguments, such as learning rate. For example:

```
from tensorflow.keras.optimizers import SGD
sgd = SGD(...)
model.compile(optimizer=sgd)
```

Listing 6.7: Assigning an optimizer to a Keras model

You can also use the default parameters of the optimizer by specifying the name of the optimizer to the optimizer argument. For example:

```
model.compile(optimizer='sgd')
```

Listing 6.8: Using SGD optimizer via its name

Some popular gradient descent optimizers you might want to choose from include:

- ▷ **sgd**: stochastic gradient descent, with support for momentum
- ▷ **rmsprop**: adaptive learning rate optimization method proposed by Geoff Hinton
- ▷ **adam**: Adaptive Moment Estimation (Adam) that also uses adaptive learning rates

You can learn about all of the optimizers supported by Keras on the *Optimizers* page.

You can learn more about different gradient descent methods in the Gradient descent optimization algorithms section of Sebastian Ruder's post *An overview of gradient descent optimization algorithms*.

6.4.1 Model Loss Functions

The loss function, also called the objective function, is the evaluation of the model used by the optimizer to navigate the weight space. You can specify the name of the loss function to use in the compile function by the loss argument. Some common examples include:

- ▷ **mean_squared_error**: for mean squared error
- ▷ **binary_crossentropy**: for binary logarithmic loss (logloss)
- ▷ **categorical_crossentropy**: for multiclass logarithmic loss (logloss)

You can learn more about the loss functions supported by Keras on the *Losses* page.

Model Metrics

Metrics are evaluated by the model during training. Usually one metric is supported at the moment, and that is accuracy.

6.5 Model Training

The model is trained on NumPy arrays using the `fit()` function; for example:

```
model.fit(X, y, epochs=..., batch_size=...)
```

Training both specifies the number of epochs to train on and the batch size.

- ▷ Epochs (`epochs`) refer to the number of times the model is exposed to the training dataset.
- ▷ Batch Size (`batch_size`) is the number of training instances shown to the model before a weight update is performed.

The `fit` function also allows for some basic evaluation of the model during training. You can set the `validation_split` value to hold back a fraction of the training dataset for validation to be evaluated in each epoch or provide a `validation_data` tuple of (`X`, `y`) data to evaluate.

Fitting the model returns a history object with details and metrics calculated for the model in each epoch. This can be used for graphing model performance.

6.6 Model Prediction

Once you have trained your model, you can use it to make predictions on test data or new data. You can either call the `evaluate()` or the `predict()` function from the model:

- ▷ `model.evaluate(X, y)`: To calculate the loss values for the input data
- ▷ `model.predict(X)`: To generate network output for the input data

For example, if you provided a batch of data `X` and the expected output `y`, you can use `evaluate()` to calculate the loss metric (the one you defined with `compile()` before). But for a batch of new data `X`, you can obtain the network output with `predict()`. It may not be the output you want but it will be the output of your network. For example, a classification problem will probably output a softmax vector for each sample. You will need to use `numpy.argmax()` to convert the softmax vector into class labels.

6.7 Summarize the Model

Once you are happy with your model, you can finalize it. You may wish to output a summary of your model. For example, you can display a summary of a model to your terminal screen by calling the `summary` function:

```
model.summary()
```

Listing 6.9: Printing a Keras model

You can also retrieve a summary of the model configuration using the `get_config()` function:

```
config = model.get_config()
```

Listing 6.10: Retrieve a model configuration as a Python dict

Finally, you can create an image of your model structure directly:

```
from tensorflow.keras.utils import plot_model
plot_model(model, to_file='model.png',
            show_shapes=True, show_dtype=True, show_layer_names=True,
            expand_nested=True, show_layer_activations=True)
```

Listing 6.11: Creating an image file representing a model structure

6.8 Resources

You can learn more about how to create simple neural network and deep learning models in Keras using the following resources:

Articles

Layer Weight Initializers. Keras API Reference.

<https://keras.io/api/layers/initializers/>

Layer activation functions. Keras API Reference.

<https://keras.io/api/layers/activations/>

Keras Layers API. Keras API Reference.

<https://keras.io/api/layers/>

Optimizers. Keras API Reference.

<https://keras.io/api/optimizers/>

Sebastian Ruder. *An overview of gradient descent optimization algorithms.*

<https://ruder.io/optimizing-gradient-descent/>

Losses. Keras API Reference.

<https://keras.io/api/losses/>

The Sequential class. Keras API Reference.

<https://keras.io/api/models/sequential/>

The Sequential Model. Keras Developer Guides.

https://keras.io/guides/sequential_model/

The Models API. Keras API Reference.

<https://keras.io/api/models/>

6.9 Summary

In this chapter, you discovered the Keras API that you can use to create artificial neural networks and deep learning models. Specifically, you learned about the life cycle of a Keras model, including:

- ▷ Constructing a model
- ▷ Creating and adding layers including weight initialization and activation

- ▷ Compiling models including optimization method, loss function and metrics
- ▷ Fitting models including epochs and batch size
- ▷ Model predictions
- ▷ Summarizing the model

You now know the basics of neural network models. In the next chapter you will develop your very first multilayer perceptron model in Keras.

Develop Your First Neural Network with Keras

Keras is a powerful and easy-to-use free open source Python library for developing and evaluating *deep learning models*. It is part of the TensorFlow library and allows you to define and train neural network models in just a few lines of code. In this chapter, you will discover how to create your first deep learning neural network model in Python using Keras. After completing this chapter you will know:

- ▷ How to load a CSV dataset ready for use with Keras.
- ▷ How to define and compile a Multilayer Perceptron model in Keras.
- ▷ How to evaluate a Keras model on a validation dataset.

Let's get started.

Overview

There is not a lot of code required, but we will go over it slowly so that you will know how to create your own models in the future. The steps you will learn in this chapter are as follows:

- ▷ Load Data
- ▷ Define Keras Model
- ▷ Compile Keras Model
- ▷ Fit Keras Model
- ▷ Evaluate Keras Model
- ▷ Tie It All Together
- ▷ Make Predictions

7.1 Load Data

The first step is to define the functions and classes you intend to use in this chapter. You will use the NumPy library to load your dataset and two classes from the Keras library to define your model.

The imports required are listed below.

```
# first neural network with keras
from numpy import loadtxt
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
...
```

Listing 7.1: Import statements used in this chapter

You can now load our dataset.

In this Keras tutorial, you will use the Pima Indians onset of diabetes dataset. This is a standard machine learning dataset from the UCI Machine Learning repository. It describes patient medical record data for Pima Indians and whether they had an onset of diabetes within five years.

It is a binary classification problem (onset of diabetes as 1 or not as 0). All of the input variables that describe each patient are numerical. This makes it easy to use directly with neural networks that expect numerical input and output values and is an ideal choice for our first neural network in Keras.

The dataset is available from the bundle of sample code provided with this book. You can also download it here:

- ▷ Dataset CSV File (<https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes.data.csv>)
- ▷ Dataset Details (<https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes.names>)

Download the dataset and place it in your local working directory, the same location as your Python file. Save it with the filename `pima-indians-diabetes.csv`. Take a look inside the file; you should see rows of data like the following:

```
6,148,72,35,0,33.6,0.627,50,1
1,85,66,29,0,26.6,0.351,31,0
8,183,64,0,0,23.3,0.672,32,1
1,89,66,23,94,28.1,0.167,21,0
0,137,40,35,168,43.1,2.288,33,1
...
```

Output 7.1: Samples of the Pima Indians dataset

You can now load the file as a matrix of numbers using the NumPy function `loadtxt()`. There are eight input variables and one output variable (the last column). You will be learning a model to map rows of input variables (X) to an output variable (y), which is often summarized as $y = f(X)$. The variables can be summarized as follows:

Input Variables (X):

1. Number of times pregnant
2. Plasma glucose concentration at 2 hours in an oral glucose tolerance test
3. Diastolic blood pressure (mm Hg)
4. Triceps skin fold thickness (mm)

5. 2-hour serum insulin ($\mu\text{IU}/\text{ml}$)
6. Body mass index (weight in $\text{kg}/(\text{height in m})^2$)
7. Diabetes pedigree function
8. Age (years)

Output Variables (y):

- ▷ Class variable (0 or 1)

Once the CSV file is loaded into memory, you can split the columns of data into input and output variables.

The data will be stored in a 2D array where the first dimension is rows and the second dimension is columns, e.g., (rows, columns). You can split the array into two arrays by selecting subsets of columns using the standard NumPy slice operator “:”. You can select the first eight columns from index 0 to index 7 via the slice `0:8`. You can then select the output column (the 9th variable) via index 8.

```
...
# load the dataset
dataset = loadtxt('pima-indians-diabetes.csv', delimiter=',')
# split into input (X) and output (y) variables
X = dataset[:,0:8]
y = dataset[:,8]
...
```

Listing 7.2: Load the dataset using NumPy

You are now ready to define your neural network model.

7.2 Define Keras Model

Models in Keras are defined as a sequence of layers. You create a `Sequential` model and add layers one at a time until you are happy with our network architecture. The first thing to get right is to ensure the input layer has the correct number of input features. This can be specified when creating the first layer with the `input_shape` argument and setting it to `(8,)` for presenting the eight input variables as a vector.

How do you know the number of layers and their types? This is a tricky question. There are heuristics that you can use, and often the best network structure is found through a process of trial and error experimentation. Generally, you need a network large enough to capture the structure of the problem. In this example, let’s use a fully-connected network structure with three layers.

Fully connected layers are defined using the `Dense` class. You can specify the number of neurons or nodes in the layer as the first argument and the activation function using the `activation` argument. Also, you will use the *rectified linear unit* activation function, referred to as ReLU, on the first two layers and the sigmoid function in the output layer.

It used to be the case that sigmoid and tanh activation functions were preferred for all layers. These days, better performance is achieved using the ReLU activation function. Using

a sigmoid on the output layer ensures your network output is between 0 and 1, and is easy to map to either a probability of class 1 or snap to a hard classification of either class with a default threshold of 0.5. You can piece it all together by adding each layer:

- ▷ The model expects rows of data with 8 variables (the `input_shape=(8,)` argument).
- ▷ The first hidden layer has 12 nodes and uses the `relu` activation function.
- ▷ The second hidden layer has 8 nodes and uses the `relu` activation function.
- ▷ The output layer has one node and uses the `sigmoid` activation function.

```
...  
# define the keras model  
model = Sequential()  
model.add(Dense(12, input_shape=(8,), activation='relu'))  
model.add(Dense(8, activation='relu'))  
model.add(Dense(1, activation='sigmoid'))  
...
```

Listing 7.3: Define the neural network model in Keras

You are free to change the design and see if you get a better or worse result than the subsequent part of this chapter. The figure below provides a depiction of the network structure:

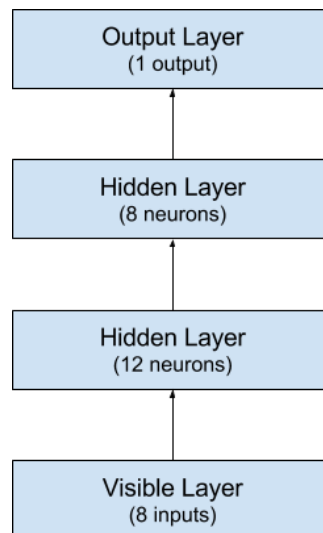


Figure 7.1: Model of a simple neural network



Note: The most confusing thing here is that the shape of the input to the model is defined as an argument on the first hidden layer. This means that the line of code that adds the first `Dense` layer is doing two things, defining the input or visible layer and the first hidden layer.

7.3 Compile Keras Model

Now that the model is defined, you can compile it. Compiling the model uses TensorFlow as an efficient numerical libraries (also called the backend). The backend automatically chooses the best way to represent the network for training and making predictions to run on your hardware, such as CPU, GPU, or even distributed. When compiling, you must specify some additional properties required when training the network. Remember training a network means finding the best set of weights to map inputs to outputs in your dataset.

You must specify the loss function to use to evaluate a set of weights, the optimizer is used to search through different weights for the network, and any optional metrics you want to collect and report during training. In this case, use cross-entropy as the `loss` argument. This loss is for a binary classification problems and is defined in Keras as “`binary_crossentropy`”. You will define the `optimizer` as the efficient stochastic gradient descent algorithm “`adam`”. This is a popular version of gradient descent because it automatically tunes itself and gives good results in a wide range of problems. Finally, because it is a classification problem, you will collect and report the classification accuracy defined via the `metrics` argument.

```
...  
# compile the keras model  
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])  
...
```

Listing 7.4: Compile the neural network model

7.4 Fit Keras Model

You have defined your model and compiled it to get ready for efficient computation. Now it is time to execute the model on some data. You can train or fit your model on your loaded data by calling the `fit()` function on the model.

Training occurs over epochs, and each epoch is split into batches.

- ▷ **Epoch:** One pass through all of the rows in the training dataset
- ▷ **Batch:** One or more samples considered by the model within an epoch before weights are updated

One epoch comprises of one or more batches, based on the chosen batch size, and the model is fit for many epochs. The training process will run for a fixed number of epochs (iterations) through the entire dataset that you must specify using the `epochs` argument. You must also set the number of dataset rows that are considered before the model weights are updated within each epoch, called the batch size, and set using the `batch_size` argument. This problem will run for a small number of epochs (150) and use a relatively small batch size of 10. These configurations can be chosen experimentally by trial and error. You want to train the model enough so that it learns a good (or good enough) mapping of rows of input data to the output classification. The model will always have some error, but the amount of error will level out after some point for a given model configuration. This is called *model convergence*.

```
...
# fit the keras model on the dataset
model.fit(X, y, epochs=150, batch_size=10)
...
```

Listing 7.5: Fit the neural network model to the dataset

This is where the work happens on your CPU or GPU.

7.5 Evaluate Keras Model

You have trained our neural network on the entire dataset, and you can evaluate the performance of the network on the same dataset. This will only give you an idea of how well you have modeled the dataset (e.g., train accuracy), but no idea of how well the algorithm might perform on new data. This was done for simplicity, but ideally, you could separate your data into train and test datasets for training and evaluation of your model.

You can evaluate your model on your training dataset using the `evaluate()` function and pass it the same input and output used to train the model. This will generate a prediction for each input and output pair and collect scores, including the average loss and any metrics you have configured, such as accuracy. The `evaluate()` function will return a list with two values. The first will be the loss of the model on the dataset, and the second will be the accuracy of the model on the dataset. You are only interested in reporting the accuracy so ignore the loss value.

```
...
# evaluate the keras model
_, accuracy = model.evaluate(X, y)
print('Accuracy: %.2f' % (accuracy*100))
```

Listing 7.6: Evaluate the neural network model on the dataset

7.6 Tie It All Together

You have just seen how you can easily create your first neural network model in Keras. Let's tie it all together into a complete code example.

```
# first neural network with keras tutorial
from numpy import loadtxt
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
# load the dataset
dataset = loadtxt('pima-indians-diabetes.csv', delimiter=',')
# split into input (X) and output (y) variables
X = dataset[:,0:8]
y = dataset[:,8]
# define the keras model
model = Sequential()
model.add(Dense(12, input_shape=(8,), activation='relu'))
```

```

model.add(Dense(8, activation='relu'))
model.add(Dense(1, activation='sigmoid'))
# compile the keras model
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
# fit the keras model on the dataset
model.fit(X, y, epochs=150, batch_size=10)
# evaluate the keras model
_, accuracy = model.evaluate(X, y)
print('Accuracy: %.2f' % (accuracy*100))

```

Listing 7.7: Complete working example of your first neural network in Keras

You can copy all the code into your Python file and save it as “keras_first_network.py” in the same directory as your data file “pima-indians-diabetes.csv”. You can then run the Python file as a script from your command line (command prompt) as follows:

```
python keras_first_network.py
```

Listing 7.8: Running a Python script

Running this example, you should see a message for each of the 150 epochs, printing the loss and accuracy, followed by the final evaluation of the trained model on the training dataset. It takes about 10 seconds to execute on my workstation running on the CPU. Ideally, you would like the loss to go to zero and the accuracy to go to 1.0 (e.g., 100%). This is not possible for any but the most trivial machine learning problems. Instead, you will always have some error in your model. The goal is to choose a model configuration and training configuration that achieve the lowest loss and highest accuracy possible for a given dataset.



Note: Your results may vary given the stochastic nature of the algorithm or evaluation procedure, or differences in numerical precision. Consider running the example a few times and compare the average outcome.

```

...
77/77 [=====] - 0s 334us/step - loss: 0.4753 - accuracy: 0.7630
Epoch 147/150
77/77 [=====] - 0s 336us/step - loss: 0.4794 - accuracy: 0.7565
Epoch 148/150
77/77 [=====] - 0s 330us/step - loss: 0.4786 - accuracy: 0.7630
Epoch 149/150
77/77 [=====] - 0s 327us/step - loss: 0.4777 - accuracy: 0.7669
Epoch 150/150
77/77 [=====] - 0s 337us/step - loss: 0.4806 - accuracy: 0.7721
24/24 [=====] - 0s 368us/step - loss: 0.4675 - accuracy: 0.7786
Accuracy: 77.86

```

Output 7.2: Output of running your first neural network in Keras

Neural networks are stochastic algorithms, meaning that the same algorithm on the same data can train a different model with different skill each time the code is run. This is a feature, not a bug. The variance in the performance of the model means that to get a reasonable approximation of how well your model is performing, you may need to fit it many times and

calculate the average of the accuracy scores. For example, below are the accuracy scores from re-running the example five times:

```
Accuracy: 75.00
Accuracy: 77.73
Accuracy: 77.60
Accuracy: 78.12
Accuracy: 76.17
```

Output 7.3: Output of running the model five different times

You can see that all accuracy scores are around 77%, and the average is 76.924%.

7.7 Make Predictions

You can adapt the above example and use it to generate predictions on the training dataset, pretending it is a new dataset you have not seen before. Making predictions is as easy as calling the `predict()` function on the model. You are using a sigmoid activation function on the output layer, so the predictions will be a probability in the range between 0 and 1. You can easily convert them into a crisp binary prediction for this classification task by rounding them. For example:

```
...
# make probability predictions with the model
predictions = model.predict(X)
# round predictions
rounded = [round(x[0]) for x in predictions]
```

Listing 7.9: Example of predicting probabilities for each example

Alternately, you can convert the probability into 0 or 1 to predict crisp classes directly; for example:

```
...
# make class predictions with the model
predictions = (model.predict(X) > 0.5).astype(int)
```

Listing 7.10: Example of prediction class labels for each example

The complete example below makes predictions for each example in the dataset, then prints the input data, predicted class, and expected class for the first five examples in the dataset.

```
# first neural network with keras make predictions
from numpy import loadtxt
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
# load the dataset
dataset = loadtxt('pima-indians-diabetes.csv', delimiter=',')
# split into input (X) and output (y) variables
X = dataset[:,0:8]
y = dataset[:,8]
```

```
# define the keras model
model = Sequential()
model.add(Dense(12, input_shape=(8,), activation='relu'))
model.add(Dense(8, activation='relu'))
model.add(Dense(1, activation='sigmoid'))
# compile the keras model
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
# fit the keras model on the dataset
model.fit(X, y, epochs=150, batch_size=10, verbose=0)
# make class predictions with the model
predictions = (model.predict(X) > 0.5).astype(int)
# summarize the first 5 cases
for i in range(5):
    print('%s => %d (expected %d)' % (X[i].tolist(), predictions[i], y[i]))
```

Listing 7.11: Complete working example of fitting a model in Keras and using it to make predictions

Running the example does not show the progress bar as before, as the verbose argument has been set to 0. After the model is fit, predictions are made for all examples in the dataset, and the input rows and predicted class value for the first five examples is printed and compared to the expected class value. You can see that most rows are correctly predicted. In fact, you can expect about 76.9% of the rows to be correctly predicted based on your estimated performance of the model in the previous section.

```
[6.0, 148.0, 72.0, 35.0, 0.0, 33.6, 0.627, 50.0] => 0 (expected 1)
[1.0, 85.0, 66.0, 29.0, 0.0, 26.6, 0.351, 31.0] => 0 (expected 0)
[8.0, 183.0, 64.0, 0.0, 0.0, 23.3, 0.672, 32.0] => 1 (expected 1)
[1.0, 89.0, 66.0, 23.0, 94.0, 28.1, 0.167, 21.0] => 0 (expected 0)
[0.0, 137.0, 40.0, 35.0, 168.0, 43.1, 2.288, 33.0] => 1 (expected 1)
```

Output 7.4: Output of making predictions with Keras



Note: Your results may vary given the stochastic nature of the algorithm or evaluation procedure, or differences in numerical precision. Consider running the example a few times and compare the average outcome.

7.8 Further Reading

Are you looking for some more Deep Learning tutorials with Python and Keras? Take a look at some of these:

Books

Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.

<https://www.amazon.com/dp/0262035618>

(Online version at <http://www.deeplearningbook.org>).

APIs

Keras official homepage.

<https://keras.io>

Keras API Reference.

<https://keras.io/api/>

7.9 Summary

In this chapter, you discovered how to create your first neural network model using the powerful Keras Python library for deep learning. Specifically, you learned the six key steps in using Keras to create a neural network or deep learning model step-by-step, including:

- ▷ How to load data
- ▷ How to define a neural network in Keras
- ▷ How to compile a Keras model using the efficient numerical backend
- ▷ How to train a model on data
- ▷ How to evaluate a model on data
- ▷ How to make predictions with the model

Now you have your first Keras model created and you will repeat this work flow for all deep learning projects. In the next chapter, we will look closer into the evaluation step.

Evaluate the Performance of Deep Learning Models

8

There are a lot of decisions to make when designing and configuring your deep learning models. Most of these decisions must be resolved empirically through trial and error and evaluating them on real data. As such, it is critically important to have a robust way to evaluate the performance of your neural networks and deep learning models. In this chapter, you will discover a few ways to evaluate model performance using Keras. After completing this lesson, you will know:

- ▷ How to evaluate a Keras model using an automatic verification dataset.
- ▷ How to evaluate a Keras model using a manual verification dataset.
- ▷ How to evaluate a Keras model using k -fold cross-validation.

Let's get started.

Overview

This chapter is in three parts; they are:

- ▷ Empirically Evaluate Network Configurations
- ▷ Data Splitting
- ▷ Manual k -Fold Cross-Validation

8.1 Empirically Evaluate Network Configurations

You must make a myriad of decisions when designing and configuring your deep learning models. Many of these decisions can be resolved by copying the structure of other people's networks and using heuristics. Ultimately, the best technique is to actually design small experiments and empirically evaluate problems using real data. This includes high-level decisions like the number, size, and type of layers in your network. It also includes lower-level decisions like the choice of the loss function, activation functions, optimization procedure, and the number of epochs.

Deep learning is often used on problems that have very large datasets. That is tens of thousands or hundreds of thousands of instances. As such, you need to have a robust test

harness that allows you to estimate the performance of a given configuration on unseen data and reliably compare the performance to other configurations.

8.2 Data Splitting

The large amount of data and the complexity of the models require very long training times. As such, it is typical to separate data into training and test datasets or training and validation datasets. Keras provides two convenient ways of evaluating your deep learning algorithms this way:

1. Use an automatic verification dataset
2. Use a manual verification dataset

Use an Automatic Verification Dataset

Keras can separate a portion of your training data into a validation dataset and evaluate the performance of your model on that validation dataset in each epoch. You can do this by setting the `validation_split` argument on the `fit()` function to a percentage of the size of your training dataset. For example, a reasonable value might be 0.2 or 0.33 for 20% or 33% of your training data held back for validation. The example below demonstrates the use of an automatic validation dataset on a small binary classification problem using the Pima Indians onset of diabetes dataset (see Section 7.1).

```
# MLP with automatic validation set
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
import numpy
# fix random seed for reproducibility
numpy.random.seed(7)
# load pima indians dataset
dataset = numpy.loadtxt("pima-indians-diabetes.csv", delimiter=",")
# split into input (X) and output (Y) variables
X = dataset[:,0:8]
Y = dataset[:,8]
# create model
model = Sequential()
model.add(Dense(12, input_shape=(8,), activation='relu'))
model.add(Dense(8, activation='relu'))
model.add(Dense(1, activation='sigmoid'))
# Compile model
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
# Fit the model
model.fit(X, Y, validation_split=0.33, epochs=150, batch_size=10)
```

Listing 8.1: Evaluate a neural network using an automatic validation set



Note: Your results may vary given the stochastic nature of the algorithm or evaluation procedure, or differences in numerical precision. Consider running the example a few times and compare the average outcome.

Running the example, you can see that the verbose output on each epoch shows the loss and accuracy on both the training dataset and the validation dataset.

```
...
Epoch 145/150
514/514 [=====] - 0s - loss: 0.5252 - acc: 0.7335 - val_loss: 0.5489 - val_
Epoch 146/150
514/514 [=====] - 0s - loss: 0.5198 - acc: 0.7296 - val_loss: 0.5918 - val_
Epoch 147/150
514/514 [=====] - 0s - loss: 0.5175 - acc: 0.7335 - val_loss: 0.5365 - val_
Epoch 148/150
514/514 [=====] - 0s - loss: 0.5219 - acc: 0.7354 - val_loss: 0.5414 - val_
Epoch 149/150
514/514 [=====] - 0s - loss: 0.5089 - acc: 0.7432 - val_loss: 0.5417 - val_
Epoch 150/150
514/514 [=====] - 0s - loss: 0.5148 - acc: 0.7490 - val_loss: 0.5549 - val_
```

Output 8.1: Output of evaluating a neural network using an automatic validation set

Use a Manual Verification Dataset

Keras also allows you to manually specify the dataset to use for validation during training. In this example, you can use the handy `train_test_split()` function from the Python scikit-learn machine learning library to separate your data into a training and test dataset. Use 67% for training and the remaining 33% of the data for validation. The validation dataset can be specified to the `fit()` function in Keras by the `validation_data` argument. It takes a tuple of the input and output datasets.

```
# MLP with manual validation set
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from sklearn.model_selection import train_test_split
import numpy
# fix random seed for reproducibility
seed = 7
numpy.random.seed(seed)
# load pima indians dataset
dataset = numpy.loadtxt("pima-indians-diabetes.csv", delimiter=",")
# split into input (X) and output (Y) variables
X = dataset[:,0:8]
Y = dataset[:,8]
# split into 67% for train and 33% for test
X_train, X_test, y_train, y_test = train_test_split(X, Y,
                                                    test_size=0.33, random_state=seed)

# create model
model = Sequential()
model.add(Dense(12, input_shape=(8,), activation='relu'))
model.add(Dense(8, activation='relu'))
model.add(Dense(1, activation='sigmoid'))
# Compile model
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
```

```
# Fit the model
model.fit(X_train, y_train, validation_data=(X_test,y_test), epochs=150, batch_size=10)
```

Listing 8.2: Evaluate a neural network using a manual validation set

Like before, running the example provides verbose output of training that includes the loss and accuracy of the model on both the training and validation datasets for each epoch.



Note: Your results may vary given the stochastic nature of the algorithm or evaluation procedure, or differences in numerical precision. Consider running the example a few times and compare the average outcome.

```
...
Epoch 145/150
514/514 [=====] - 0s - loss: 0.4847 - acc: 0.7704 - val_loss: 0.5668 - val_
Epoch 146/150
514/514 [=====] - 0s - loss: 0.4853 - acc: 0.7549 - val_loss: 0.5768 - val_
Epoch 147/150
514/514 [=====] - 0s - loss: 0.4864 - acc: 0.7743 - val_loss: 0.5604 - val_
Epoch 148/150
514/514 [=====] - 0s - loss: 0.4831 - acc: 0.7665 - val_loss: 0.5589 - val_
Epoch 149/150
514/514 [=====] - 0s - loss: 0.4961 - acc: 0.7782 - val_loss: 0.5663 - val_
Epoch 150/150
514/514 [=====] - 0s - loss: 0.4967 - acc: 0.7588 - val_loss: 0.5810 - val_
```

Output 8.2: Output of evaluating a neural network using an manual validation set

8.3 Manual k -Fold Cross-Validation

The gold standard for machine learning model evaluation is k -fold cross-validation. It provides a robust estimate of the performance of a model on unseen data. It does this by splitting the training dataset into k subsets, taking turns training models on all subsets except one, which is held out, and evaluating model performance on the held-out validation dataset. The process is repeated until all subsets are given an opportunity to be the held-out validation set. The performance measure is then averaged across all models that are created.

It is important to understand that cross-validation means estimating a model design (e.g., 3-layer vs. 4-layer neural network) rather than a specific fitted model. You do not want to use a specific dataset to fit the models and compare the result since this may be due to that particular dataset fitting better on one model design (as known as *overfitting*). Instead, you want to use multiple datasets to fit, resulting in multiple fitted models of the same design, taking the average performance measure for comparison.

Cross-validation is often not used for evaluating deep learning models because of the greater computational expense. For example, k -fold cross-validation is often used with 5 or 10 folds. As such, 5 or 10 models must be constructed and evaluated, significantly adding to the evaluation time of a model. Nevertheless, when the problem is small enough or if you have

sufficient computing resources, *k*-fold cross-validation can give you a less-biased estimate of the performance of your model.

In the example below, you will use the handy `StratifiedKFold` class from the scikit-learn Python machine learning library to split the training dataset into 10 folds. The folds are stratified, meaning that the algorithm attempts to balance the number of instances of each class in each fold. The example creates and evaluates 10 models using the 10 splits of the data and collects all the scores. The verbose output for each epoch is turned off by passing `verbose=0` to the `fit()` and `evaluate()` functions on the model. The performance is printed for each model, and it is stored. The average and standard deviation of the model performance are then printed at the end of the run to provide a robust estimate of model accuracy.

```
# MLP for Pima Indians Dataset with 10-fold cross validation
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from sklearn.model_selection import StratifiedKFold
import numpy as np
# fix random seed for reproducibility
seed = 7
np.random.seed(seed)
# load pima indians dataset
dataset = np.loadtxt("pima-indians-diabetes.csv", delimiter=",")
# split into input (X) and output (Y) variables
X = dataset[:,0:8]
Y = dataset[:,8]
# define 10-fold cross validation test harness
kfold = StratifiedKFold(n_splits=10, shuffle=True, random_state=seed)
cvscores = []
for train, test in kfold.split(X, Y):
    # create model
    model = Sequential()
    model.add(Dense(12, input_shape=(8,), activation='relu'))
    model.add(Dense(8, activation='relu'))
    model.add(Dense(1, activation='sigmoid'))
    # Compile model
    model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
    # Fit the model
    model.fit(X[train], Y[train], epochs=150, batch_size=10, verbose=0)
    # evaluate the model
    scores = model.evaluate(X[test], Y[test], verbose=0)
    print("%s: %.2f%%" % (model.metrics_names[1], scores[1]*100))
    cvscores.append(scores[1] * 100)

print("%.2f%% (+/- %.2f%%)" % (np.mean(cvscores), np.std(cvscores)))
```

Listing 8.3: Evaluate a neural network using scikit-learn



Note: Your results may vary given the stochastic nature of the algorithm or evaluation procedure, or differences in numerical precision. Consider running the example a few times and compare the average outcome.

Running the example will take less than a minute and will produce the following output:

```
acc: 77.92%  
acc: 68.83%  
acc: 72.73%  
acc: 64.94%  
acc: 77.92%  
acc: 35.06%  
acc: 74.03%  
acc: 68.83%  
acc: 34.21%  
acc: 72.37%  
64.68% (+/- 15.50%)
```

Output 8.3: Output of evaluating a neural network using scikit-learn

Notice that we had to re-create the model each loop then fit and evaluate it with the data for the fold. In chapter 13, we will look at how we can use Keras models natively with the scikit-learn machine learning library.

8.4 Summary

In this chapter, you discovered the importance of having a robust way to estimate the performance of your deep learning models on unseen data. You learned three ways that you can estimate the performance of your deep learning models in Python using the Keras library:

- ▷ Automatically splitting a training dataset into train and validation datasets
- ▷ Manually and explicitly defining a training and validation dataset
- ▷ Evaluating performance using k -fold cross-validation, the gold standard technique

In the next chapter, we will see another example of using Keras to build a neural network.

Project: Multiclass Classification of Iris Species

In this chapter, you will discover how to use Keras to develop and evaluate neural network models for multiclass classification problems. After completing this chapter, you will know:

- ▷ How to load data from CSV and make it available to Keras
- ▷ How to prepare multiclass classification data for modeling with neural networks
- ▷ How to evaluate Keras neural network models with scikit-learn

Let's get started.

Overview

This chapter is in six parts; they are:

- ▷ Problem Description
- ▷ Import Classes and Functions
- ▷ Load the Dataset
- ▷ Encode the Output Variable
- ▷ Evaluate the Model with k -Fold Cross-Validation
- ▷ Complete Example

9.1 Problem Description

In this chapter, you will use the standard machine learning problem called the iris flowers dataset. This dataset is well studied and makes a good problem for practicing on neural networks because all four input variables are numeric and have the same scale in centimeters. Each instance describes the properties of an observed flower's measurements, and the output variable is a specific iris species. The attributes for this dataset can be summarized as follows:

1. Sepal length in centimeters
2. Sepal width in centimeters
3. Petal length in centimeters

4. Petal width in centimeters
5. Class (the flower species)

This is a multiclass classification problem, meaning that there are more than two classes to be predicted. In fact, there are three flower species. This is an important problem for practicing with neural networks because the three class values require specialized handling. Below is a sample of the first five of the 150 instances:

```
5.1,3.5,1.4,0.2,Iris-setosa
4.9,3.0,1.4,0.2,Iris-setosa
4.7,3.2,1.3,0.2,Iris-setosa
4.6,3.1,1.5,0.2,Iris-setosa
5.0,3.6,1.4,0.2,Iris-setosa
```

Output 9.1: Sample of the iris flowers dataset

The iris flower dataset is a well-studied problem, and as such, you can expect to achieve a model accuracy in the range of 95% to 97%. This provides a good target to aim for when developing your models. The dataset is provided in the bundle of sample code provided with this book. You can also download the iris flowers dataset from the UCI Machine Learning repository and place it in your current working directory with the filename `iris.csv`. You can learn more about the iris flower classification dataset on the UCI Machine Learning Repository page.

9.2 Import Classes and Functions

You can begin by importing all the classes and functions you will need in this chapter. This includes both the functionality you require from Keras and the data loading from pandas, as well as data preparation and model evaluation from scikit-learn and SciKeras.

```
import pandas as pd
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from scikeras.wrappers import KerasClassifier
from tensorflow.keras.utils import to_categorical
from sklearn.model_selection import cross_val_score
from sklearn.model_selection import KFold
from sklearn.preprocessing import LabelEncoder
from sklearn.pipeline import Pipeline
...
```

Listing 9.1: Import classes and functions

9.3 Load the Dataset

The dataset can be loaded directly. Because the output variable contains strings, it is easiest to load the data using pandas. You can then split the attributes (columns) into input variables (X) and output variables (Y).


```
...
# load dataset
dataframe = pd.read_csv("iris.csv", header=None)
dataset = dataframe.values
X = dataset[:,0:4].astype(float)
Y = dataset[:,4]
```

Listing 9.2: Load dataset and separate into input and output variables

9.4 Encode the Output Variable

The output variable contains three different string values. When modeling multiclass classification problems using neural networks, it is good practice to reshape the output attribute from a vector of class labels to a matrix of a Boolean for each class and whether a given instance is in that class. This is called one-hot encoding or creating dummy variables from a categorical variable. For example, in this problem, the three class values are *Iris-setosa*, *Iris-versicolor*, and *Iris-virginica*. If you had the three observations:

```
Iris-setosa
Iris-versicolor
Iris-virginica
```

Output 9.2: Three classes in the iris dataset

You can turn this into a one-hot encoded binary matrix for each data instance that would look like this:

Iris-setosa,	Iris-versicolor,	Iris-virginica
1,	0,	0
0,	1,	0
0,	0,	1

Output 9.3: One-hot encoding of the classes in the iris dataset

You can first encode the strings consistently to integers using the scikit-learn class `LabelEncoder`. Then convert the vector of integers to a one-hot encoding using the Keras function `to_categorical()`.

```
...
# encode class values as integers
encoder = LabelEncoder()
encoder.fit(Y)
encoded_Y = encoder.transform(Y)
# convert integers to dummy variables (i.e. one-hot encoded)
dummy_y = to_categorical(encoded_Y)
```

Listing 9.3: One-hot encoding of iris dataset output variable

9.5 Define the Neural Network Model

The Keras library provides wrapper classes to allow you to use neural network models developed with Keras in scikit-learn. There is a `KerasClassifier` class in `SciKeras` that can be used as an estimator in scikit-learn, the base type of model in the library. The `KerasClassifier` takes the name of a function as an argument. This function must return the constructed neural network model, ready for training.

Below is a function that will create a baseline neural network for the iris classification problem. It creates a simple, fully connected network with one hidden layer that contains eight neurons. The hidden layer uses a rectifier activation function which is a good practice. Because you used a one-hot encoding for your iris dataset, the output layer must create three output values, one for each class. The output value with the largest value will be taken as the class predicted by the model. The network topology of this simple one-layer neural network can be summarized as follows:

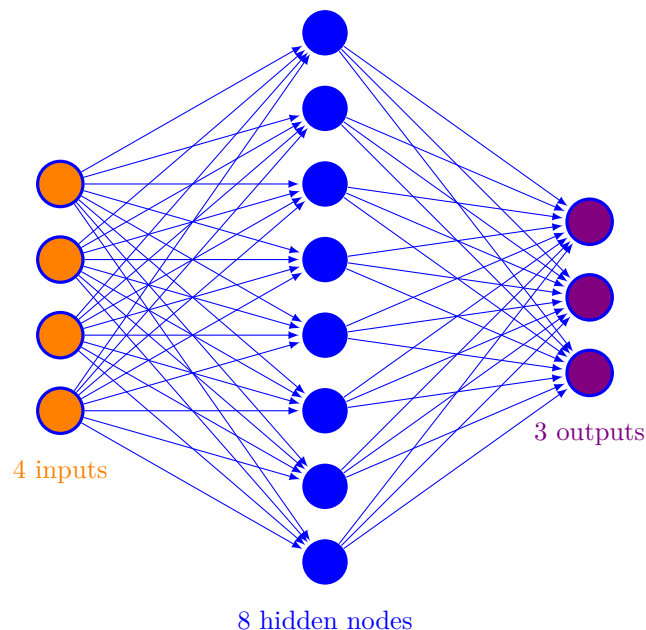


Figure 9.1: Example network structure

Note that a “softmax” activation function was used in the output layer. This ensures the output values are in the range of 0 and 1 and may be used as predicted probabilities. Finally, the network uses the efficient Adam gradient descent optimization algorithm with a logarithmic loss function, which is called “categorical_crossentropy” in Keras.

```
...
# define baseline model
def baseline_model():
    # create model
    model = Sequential()
    model.add(Dense(8, input_shape=(4,), activation='relu'))
    model.add(Dense(3, activation='softmax'))
    # Compile model
```

```
model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
return model
```

Listing 9.4: Define and compile the neural network model

You can now create your `KerasClassifier` for use in scikit-learn. You can also pass arguments in the construction of the `KerasClassifier` class that will be passed on to the `fit()` function internally used to train the neural network. Here, you pass the number of epochs as 200 and batch size as 5 to use when training the model. Debugging is also turned off when training by setting `verbose` to 0.

```
...
estimator = KerasClassifier(model=baseline_model, epochs=200, batch_size=5, verbose=0)
```

Listing 9.5: Create wrapper for neural network model for use in scikit-learn

9.6 Evaluate the Model with k -Fold Cross-Validation

You can now evaluate the neural network model on our training data. The scikit-learn library has excellent capability to evaluate models using a suite of techniques. The gold standard for evaluating machine learning models is k -fold cross-validation. First, define the model evaluation procedure. Here, you set the number of folds to 10 (an excellent default) and shuffle the data before partitioning it.

```
...
kfold = KFold(n_splits=10, shuffle=True)
```

Listing 9.6: Prepare cross-validation

Now, you can evaluate your model (`estimator`) on your dataset (`X` and `dummy_y`) using a 10-fold cross-validation procedure (`KFold`). Evaluating the model only takes approximately 10 seconds and returns an object that describes the evaluation of the ten constructed models for each of the splits of the dataset.

```
...
results = cross_val_score(estimator, X, dummy_y, cv=kfold)
print("Baseline: %.2f%% (%.2f%%)" % (results.mean()*100, results.std()*100))
```

Listing 9.7: Evaluate the neural network model

9.7 Complete Example

You can tie all of this together into a single program that you can save and run as a script:

```
# multi-class classification with Keras
import pandas
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
```

```

from scikeras.wrappers import KerasClassifier
from tensorflow.keras.utils import to_categorical
from sklearn.model_selection import cross_val_score
from sklearn.model_selection import KFold
from sklearn.preprocessing import LabelEncoder
from sklearn.pipeline import Pipeline
# load dataset
dataframe = pandas.read_csv("iris.csv", header=None)
dataset = dataframe.values
X = dataset[:,0:4].astype(float)
Y = dataset[:,4]
# encode class values as integers
encoder = LabelEncoder()
encoder.fit(Y)
encoded_Y = encoder.transform(Y)
# convert integers to dummy variables (i.e. one-hot encoded)
dummy_y = to_categorical(encoded_Y)

# define baseline model
def baseline_model():
    # create model
    model = Sequential()
    model.add(Dense(8, input_shape=(4,), activation='relu'))
    model.add(Dense(3, activation='softmax'))
    # Compile model
    model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
    return model

estimator = KerasClassifier(model=baseline_model, epochs=200, batch_size=5, verbose=0)
kfold = KFold(n_splits=10, shuffle=True)
results = cross_val_score(estimator, X, dummy_y, cv=kfold)
print("Baseline: %.2f%% (%.2f%%)" % (results.mean()*100, results.std()*100))

```

Listing 9.8: Multilayer perceptron model for iris flower problem

The results are summarized as both the mean and standard deviation of the model accuracy on the dataset. This is a reasonable estimation of the performance of the model on unseen data. It is also within the realm of known top results for this problem.



Note: Your results may vary given the stochastic nature of the algorithm or evaluation procedure, or differences in numerical precision. Consider running the example a few times and compare the average outcome.

Accuracy: 97.33% (4.42%)

Output 9.4: Estimated accuracy of neural network model on the Iris dataset

9.8 Further Readings

This section provides more resources on the topic if you are looking to go deeper.

Online Resources

Iris Data Set. UCI Machine Learning Repository.

<https://archive.ics.uci.edu/ml/datasets/Iris>

SciKeras documentation.

<https://www.adriangb.com/scikeras/stable/>

KFold cross-validator. scikit-learn documentation.

https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.KFold.html

9.9 Summary

In this chapter, you discovered how to develop and evaluate a neural network using the Keras Python library for deep learning. By completing this chapter, you learned:

- ▷ How to load data and make it available to Keras
- ▷ How to prepare multiclass classification data for modeling using one-hot encoding
- ▷ How to use Keras neural network models with scikit-learn
- ▷ How to define a neural network using Keras for multiclass classification
- ▷ How to evaluate a Keras neural network model using scikit-learn with k -fold cross-validation

In the next chapter, you will see how Keras can be used for classification of only two classes.

Project: Binary Classification of Sonar Returns

10

In this chapter you will discover how to effectively use the Keras library in your machine learning project by working through a binary classification project step-by-step. After completing this chapter, you will know:

- ▷ How to load training data and make it available to Keras
- ▷ How to design and train a neural network for tabular data
- ▷ How to evaluate the performance of a neural network model in Keras on unseen data
- ▷ How to perform data preparation to improve skill when using neural networks
- ▷ How to tune the topology and configuration of neural networks in Keras

Let's get started.

Overview

This chapter is in four parts; they are:

- ▷ Sonar Object Classification Dataset
- ▷ Baseline Neural Network Model Performance
- ▷ Improve the Baseline Model with Data Preparation
- ▷ Tuning Layers and Number of Neurons in the Model

10.1 Sonar Object Classification Dataset

The dataset you will use in this chapter is the Sonar dataset. This is a dataset that describes sonar chirp returns bouncing off different surfaces. The 60 input variables are the strength of the returns at different angles. It is a binary classification problem that requires a model to differentiate rocks from metal cylinders.

It is a well-understood dataset. All the variables are continuous and generally in the range of 0 to 1. The output variable is a string “M” for mine and “R” for rock, which will need to be converted to integers 1 and 0 as neural networks can only output numbers. The dataset

is in the bundle of source code provided with this book. Alternatively, you can download¹ the dataset and place it in your working directory with the filename `sonar.csv`.

A benefit of using this dataset is that it is a standard benchmark problem. This means that we have some idea of the expected skill of a good model. Using cross-validation, a neural network should be able to achieve a performance of around 84% with an upper bound on accuracy for custom models at around 88%. You can learn more about this dataset on the UCI Machine Learning repository.

10.2 Baseline Neural Network Model Performance

Let's create a baseline model and result for this problem. You will start by importing all the classes and functions you will need.

```
import pandas as pd
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from scikeras.wrappers import KerasClassifier
from sklearn.model_selection import cross_val_score
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
...
```

Listing 10.1: Import classes and functions

Now, you can load the dataset using Pandas and split the columns into 60 input variables (X) and one output variable (Y). Use Pandas to load the data because it easily handles strings (the output variable), whereas attempting to load the data directly using NumPy would be more difficult.

```
...
# load dataset
dataframe = pd.read_csv("sonar.csv", header=None)
dataset = dataframe.values
# split into input (X) and output (Y) variables
X = dataset[:,0:60].astype(float)
Y = dataset[:,60]
```

Listing 10.2: Load the dataset and separate into input and output variables

The output variable is string values. You must convert them into integer values 0 and 1. You can do this using the `LabelEncoder` class from scikit-learn. This class will model the encoding required using the entire dataset via the `fit()` function, then apply the encoding to create a new output variable using the `transform()` function.

¹<https://raw.githubusercontent.com/jbrownlee/Datasets/master/sonar.csv>

```
...
# encode class values as integers
encoder = LabelEncoder()
encoder.fit(Y)
encoded_Y = encoder.transform(Y)
```

Listing 10.3: Label encode output variables

You are now ready to create your neural network model using Keras. You will use scikit-learn to evaluate the model using stratified k -fold cross-validation. This is a resampling technique that will provide an estimate of the performance of the model. It does this by splitting the data into k parts and training the model on all parts except one, which is held out as a test set to evaluate the performance of the model. This process is repeated k times, and the average score across all constructed models is used as a robust estimate of performance. It is stratified, meaning that it will look at the output values and attempt to balance the number of instances that belong to each class in the k splits of the data.

To use Keras models with scikit-learn, you must use the `KerasClassifier` wrapper from the `SciKeras` module. This class takes a function that creates and returns our neural network model. It also takes arguments that it will pass along to the call to `fit()`, such as the number of epochs and the batch size.

Let's start by defining the function that creates your baseline model. Your model will have a single, fully connected hidden layer with the same number of neurons as input variables. This is a good default starting point when creating neural networks.

The weights are initialized using a small Gaussian random number. The Rectifier activation function is used. The output layer contains a single neuron in order to make predictions. It uses the sigmoid activation function in order to produce a probability output in the range of 0 to 1 that can easily and automatically be converted to crisp class values.

Finally, you will use the logarithmic loss function (`binary_crossentropy`) during training, the preferred loss function for binary classification problems. The model also uses the efficient Adam optimization algorithm for gradient descent, and accuracy metrics will be collected when the model is trained.

```
# baseline model
def create_baseline():
    # create model
    model = Sequential()
    model.add(Dense(60, input_shape=(60,), activation='relu'))
    model.add(Dense(1, activation='sigmoid'))
    # Compile model
    model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
    return model
```

Listing 10.4: Define and compile baseline model

Now, it is time to evaluate this model using stratified cross-validation in the scikit-learn framework. Pass the number of training epochs to the `KerasClassifier`, again using reasonable default values. Verbose output is also turned off, given that the model will be created ten times for the 10-fold cross-validation being performed.


```
...
# evaluate model with standardized dataset
estimator = KerasClassifier(model=create_baseline, epochs=100, batch_size=5, verbose=0)
kfold = StratifiedKFold(n_splits=10, shuffle=True)
results = cross_val_score(estimator, X, encoded_Y, cv=kfold)
print("Baseline: %.2f%% (%.2f%%)" % (results.mean()*100, results.std()*100))
```

Listing 10.5: Fit and evaluate baseline model

After tying this together, the complete example is listed below.

```
# Binary Classification with Sonar Dataset: Baseline
from pandas import read_csv
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from scikeras.wrappers import KerasClassifier
from sklearn.model_selection import cross_val_score
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import StratifiedKFold
# load dataset
dataframe = read_csv("sonar.csv", header=None)
dataset = dataframe.values
# split into input (X) and output (Y) variables
X = dataset[:,0:60].astype(float)
Y = dataset[:,60]
# encode class values as integers
encoder = LabelEncoder()
encoder.fit(Y)
encoded_Y = encoder.transform(Y)
# baseline model
def create_baseline():
    # create model
    model = Sequential()
    model.add(Dense(60, input_shape=(60,), activation='relu'))
    model.add(Dense(1, activation='sigmoid'))
    # Compile model
    model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
    return model
# evaluate model with standardized dataset
estimator = KerasClassifier(model=create_baseline, epochs=100, batch_size=5, verbose=0)
kfold = StratifiedKFold(n_splits=10, shuffle=True)
results = cross_val_score(estimator, X, encoded_Y, cv=kfold)
print("Baseline: %.2f%% (%.2f%%)" % (results.mean()*100, results.std()*100))
```

Listing 10.6: Multilayer perceptron model for sonar problem

Running this code produces the following output showing the mean and standard deviation of the estimated accuracy of the model on unseen data.



Note: Your results may vary given the stochastic nature of the algorithm or evaluation procedure, or differences in numerical precision. Consider running the example a few times and compare the average outcome.

```
Baseline: 81.68% (7.26%)
```

Output 10.1: Sample output from fitting and evaluating the baseline model

This is an excellent score without doing any hard work.

10.3 Improve the Baseline Model with Data Preparation

It is a good practice to prepare your data before modeling. Neural network models are especially suitable for having consistent input values, both in scale and distribution. Standardization is an effective data preparation scheme for tabular data when building neural network models. This is where the data is rescaled such that the mean value for each attribute is 0, and the standard deviation is 1. This preserves Gaussian and Gaussian-like distributions while normalizing the central tendencies for each attribute. You can use scikit-learn to perform the standardization of your sonar dataset using the `StandardScaler` class.

Rather than performing the standardization on the entire dataset, it is good practice to train the standardization procedure on the training data within the pass of a cross-validation run, and use the trained standardization instance to prepare the unseen test fold. This makes standardization a step in model preparation in the cross-validation process. It prevents the algorithm from having knowledge of unseen data during evaluation, knowledge that might be passed from the data preparation scheme like a crisper distribution.

You can achieve this in scikit-learn using a `Pipeline` class. The pipeline is a wrapper that executes one or more models within a pass of the cross-validation procedure. Here, you can define a pipeline with the `StandardScaler` followed by your neural network model.

```
...
# evaluate baseline model with standardized dataset
estimators = []
estimators.append(('standardize', StandardScaler()))
estimators.append(('mlp', KerasClassifier(model=create_baseline,
                                          epochs=100, batch_size=5, verbose=0)))

pipeline = Pipeline(estimators)
kfold = StratifiedKFold(n_splits=10, shuffle=True)
results = cross_val_score(pipeline, X, encoded_Y, cv=kfold)
print("Standardized: %.2f%% (%.2f%%)" % (results.mean()*100, results.std()*100))
```

Listing 10.7: Preprocess the sonar data with standardization

After tying this together, the complete example is listed below.

```
# Binary Classification with Sonar Dataset: Standardized
from pandas import read_csv
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from scikeras.wrappers import KerasClassifier
from sklearn.model_selection import cross_val_score
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import StratifiedKFold
from sklearn.preprocessing import StandardScaler
```

```

from sklearn.pipeline import Pipeline
# load dataset
dataframe = read_csv("sonar.csv", header=None)
dataset = dataframe.values
# split into input (X) and output (Y) variables
X = dataset[:,0:60].astype(float)
Y = dataset[:,60]
# encode class values as integers
encoder = LabelEncoder()
encoder.fit(Y)
encoded_Y = encoder.transform(Y)
# baseline model
def create_baseline():
    # create model
    model = Sequential()
    model.add(Dense(60, input_shape=(60,), activation='relu'))
    model.add(Dense(1, activation='sigmoid'))
    # Compile model
    model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
    return model
# evaluate baseline model with standardized dataset
estimators = []
estimators.append(('standardize', StandardScaler()))
estimators.append(('mlp', KerasClassifier(model=create_baseline,
                                          epochs=100, batch_size=5, verbose=0)))
pipeline = Pipeline(estimators)
kfold = StratifiedKFold(n_splits=10, shuffle=True)
results = cross_val_score(pipeline, X, encoded_Y, cv=kfold)
print("Standardized: %.2f%% (%.2f%%)" % (results.mean()*100, results.std()*100))

```

Listing 10.8: Update experiment to use data standardization

Running this example provides the results below.



Note: Your results may vary given the stochastic nature of the algorithm or evaluation procedure, or differences in numerical precision. Consider running the example a few times and compare the average outcome.

You now see a small but very nice lift in the mean accuracy.

```
Standardized: 84.56% (5.74%)
```

Output 10.2: Sample output from update using data standardization

10.4 Tuning Layers and Number of Neurons in the Model

There are many things to tune on a neural network, such as weight initialization, activation functions, optimization procedure, and so on. One aspect that may have an outsized effect is the structure of the network itself, called the network topology. In this section, you will look

at two experiments on the structure of the network: making it smaller and making it larger. These are good experiments to perform when tuning a neural network on your problem.

Evaluate a Smaller Network

Note that there is likely a lot of redundancy in the input variables for this problem. The data describes the same signal from different angles. Perhaps some of those angles are more relevant than others. So you can force a type of feature extraction by the network by restricting the representational space in the first hidden layer.

In this experiment, you will take your baseline model with 60 neurons in the hidden layer and reduce it by half to 30. This will pressure the network during training to pick out the most important structure in the input data to model.

You will also standardize the data as in the previous experiment with data preparation and try to take advantage of the slight lift in performance.

```
...
# smaller model
def create_smaller():
    # create model
    model = Sequential()
    model.add(Dense(30, input_shape=(60,), activation='relu'))
    model.add(Dense(1, activation='sigmoid'))
    # Compile model
    model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
    return model
estimators = []
estimators.append(('standardize', StandardScaler()))
estimators.append(('mlp', KerasClassifier(model=create_smaller,
                                         epochs=100, batch_size=5, verbose=0)))

pipeline = Pipeline(estimators)
kfold = StratifiedKFold(n_splits=10, shuffle=True)
results = cross_val_score(pipeline, X, encoded_Y, cv=kfold)
print("Smaller: %.2f%% (%.2f%%)" % (results.mean()*100, results.std()*100))
```

Listing 10.9: Creating a smaller network topology

After tying this together, the complete example is listed below.

```
# Binary Classification with Sonar Dataset: Standardized Smaller
from pandas import read_csv
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from scikeras.wrappers import KerasClassifier
from sklearn.model_selection import cross_val_score
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
# load dataset
dataframe = read_csv("sonar.csv", header=None)
dataset = dataframe.values
```

```

# split into input (X) and output (Y) variables
X = dataset[:,0:60].astype(float)
Y = dataset[:,60]
# encode class values as integers
encoder = LabelEncoder()
encoder.fit(Y)
encoded_Y = encoder.transform(Y)
# smaller model
def create_smaller():
    # create model
    model = Sequential()
    model.add(Dense(30, input_shape=(60,), activation='relu'))
    model.add(Dense(1, activation='sigmoid'))
    # Compile model
    model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
    return model
estimators = []
estimators.append(('standardize', StandardScaler()))
estimators.append(('mlp', KerasClassifier(model=create_smaller,
                                          epochs=100, batch_size=5, verbose=0)))

pipeline = Pipeline(estimators)
kfold = StratifiedKFold(n_splits=10, shuffle=True)
results = cross_val_score(pipeline, X, encoded_Y, cv=kfold)
print("Smaller: %.2f%% (%.2f%%)" % (results.mean()*100, results.std()*100))

```

Listing 10.10: Update to use a smaller network topology

Running this example provides the following result. You can see that you have a very slight boost in the mean estimated accuracy and an important reduction in the standard deviation (average spread) of the accuracy scores for the model. This is a great result because we are doing slightly better with a network half the size, which, in turn, takes half the time to train.



Note: Your results may vary given the stochastic nature of the algorithm or evaluation procedure, or differences in numerical precision. Consider running the example a few times and compare the average outcome.

```
Smaller: 86.04% (4.00%)
```

Output 10.3: Sample output from using a smaller network topology

Evaluate a Larger Network

A neural network topology with more layers offers more opportunities for the network to extract key features and recombine them in useful nonlinear ways. You can easily evaluate whether adding more layers to the network improves the performance by making another small tweak to the function used to create our model. Here, you add one new layer (one line) to the network that introduces another hidden layer with 30 neurons after the first hidden layer. Your network now has the topology:

```
60 inputs -> [60 -> 30] -> 1 output
```

Output 10.4: Summary of new network topology

The idea here is that the network is given the opportunity to model all input variables before being bottlenecked and forced to halve the representational capacity, much like you did in the experiment above with the smaller network. Instead of squeezing the representation of the inputs themselves, you have an additional hidden layer to aid in the process.

```
...
# larger model
def create_larger():
    # create model
    model = Sequential()
    model.add(Dense(60, input_shape=(60,), activation='relu'))
    model.add(Dense(30, activation='relu'))
    model.add(Dense(1, activation='sigmoid'))
    # Compile model
    model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
    return model
estimators = []
estimators.append(('standardize', StandardScaler()))
estimators.append(('mlp', KerasClassifier(model=create_larger,
                                          epochs=100, batch_size=5, verbose=0)))

pipeline = Pipeline(estimators)
kfold = StratifiedKFold(n_splits=10, shuffle=True)
results = cross_val_score(pipeline, X, encoded_Y, cv=kfold)
print("Larger: %.2f%% (%.2f%%)" % (results.mean()*100, results.std()*100))
```

Listing 10.11: Creating a larger network topology

After tying this together, the complete example is listed below.

```
# Binary Classification with Sonar Dataset: Standardized Larger
from pandas import read_csv
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from scikeras.wrappers import KerasClassifier
from sklearn.model_selection import cross_val_score
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

# load dataset
dataframe = read_csv("sonar.csv", header=None)
dataset = dataframe.values
# split into input (X) and output (Y) variables
X = dataset[:,0:60].astype(float)
Y = dataset[:,60]
# encode class values as integers
encoder = LabelEncoder()
encoder.fit(Y)
encoded_Y = encoder.transform(Y)
```

```
# larger model
def create_larger():
    # create model
    model = Sequential()
    model.add(Dense(60, input_shape=(60,), activation='relu'))
    model.add(Dense(30, activation='relu'))
    model.add(Dense(1, activation='sigmoid'))
    # Compile model
    model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
    return model

estimators = []
estimators.append(('standardize', StandardScaler()))
estimators.append(('mlp', KerasClassifier(model=create_larger,
                                          epochs=100, batch_size=5, verbose=0)))

pipeline = Pipeline(estimators)
kfold = StratifiedKFold(n_splits=10, shuffle=True)
results = cross_val_score(pipeline, X, encoded_Y, cv=kfold)
print("Larger: %.2f%% (%.2f%%)" % (results.mean()*100, results.std()*100))
```

Listing 10.12: Update to use a larger network topology

Running this example produces the results below. You can see that you do not get a lift in the model performance. This may be statistical noise or a sign that further training is needed.



Note: Your results may vary given the stochastic nature of the algorithm or evaluation procedure, or differences in numerical precision. Consider running the example a few times and compare the average outcome.

```
Larger: 83.14% (4.52%)
```

Output 10.5: Sample output from using a larger network topology

With further tuning of aspects like the optimization algorithm and the number of training epochs, it is expected that further improvements are possible. What is the best score that you can achieve on this dataset?

10.5 Further Reading

This section provides more resources on the topic if you are looking to go deeper.

Online Resources

Connectionist Bench (Snoar, Mines vs. Rocks) Data Set. UCI Machine Learning Repository.

[https://archive.ics.uci.edu/ml/datasets/Connectionist+Bench+\(Sonar,+Mines+vs.+Rocks\)](https://archive.ics.uci.edu/ml/datasets/Connectionist+Bench+(Sonar,+Mines+vs.+Rocks))

SciKeras documentation.

<https://www.adriangb.com/scikeras/stable/>

KFold cross-validator. scikit-learn documentation.

https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.KFold.html

10.6 Summary

In this chapter, you discovered the Keras deep learning library in Python. You learned how you can work through a binary classification problem step-by-step with Keras, specifically:

- ▷ How to load and prepare data for use in Keras
- ▷ How to create a baseline neural network model
- ▷ How to evaluate a Keras model using scikit-learn and stratified k -fold cross-validation
- ▷ How data preparation schemes can lift the performance of your models
- ▷ How experiments adjusting the network topology can lift model performance

In the next chapter, you will see Keras can build models to solve not only classification, but also regression problems.

Project: Regression of Boston House Prices

11

In this chapter, you will discover how to develop and evaluate neural network models using Keras for a regression problem. After completing this chapter, you will know:

- ▷ How to load a CSV dataset and make it available to Keras
- ▷ How to create a neural network model with Keras for a regression problem
- ▷ How to use scikit-learn with Keras to evaluate models using cross-validation
- ▷ How to perform data preparation in order to improve skill with Keras models
- ▷ How to tune the network topology of models with Keras

Let's get started.

Overview

This chapter is in four parts; they are:

- ▷ Problem Description
- ▷ Develop a Baseline Neural Network Model
- ▷ Modeling the Standardized Dataset
- ▷ Tune the Neural Network Topology

11.1 Problem Description

The problem that we will look at in this chapter is the Boston house price dataset. The dataset describes 13 numerical properties of houses in Boston suburbs and is concerned with modeling the price of houses in those suburbs (the 14th property) in thousands of dollars. As such, this is a regression predictive modeling problem. The full list of attributes in this dataset are as follows:

1. **CRIM**: per capita crime rate by town
2. **ZN**: proportion of residential land zoned for lots over 25,000 sq.ft.
3. **INDUS**: proportion of non-retail business acres per town

4. CHAS: Charles River dummy variable (= 1 if tract bounds river; 0 otherwise)
5. NOX: nitric oxides concentration (parts per 10 million)
6. RM: average number of rooms per dwelling
7. AGE: proportion of owner-occupied units built prior to 1940
8. DIS: weighted distances to five Boston employment centers
9. RAD: index of accessibility to radial highways
10. TAX: full-value property-tax rate per \$10,000
11. PTRATIO: pupil-teacher ratio by town
12. B: $1000(B_k - 0.63)^2$ where B_k is the proportion of blacks by town
13. LSTAT: % lower status of the population
14. MEDV: Median value of owner-occupied homes in \$1000s

This is a well-studied problem in machine learning. It is convenient to work with because all the input and output attributes are numerical, and there are 506 instances to work with. A sample of the first 5 of the 506 rows in the dataset is provided below:

0.00632	18.00	2.310	0	0.5380	6.5750	65.20	4.0900	1	296.0	15.30	396.90	4.98	24.00
0.02731	0.00	7.070	0	0.4690	6.4210	78.90	4.9671	2	242.0	17.80	396.90	9.14	21.60
0.02729	0.00	7.070	0	0.4690	7.1850	61.10	4.9671	2	242.0	17.80	392.83	4.03	34.70
0.03237	0.00	2.180	0	0.4580	6.9980	45.80	6.0622	3	222.0	18.70	394.63	2.94	33.40
0.06905	0.00	2.180	0	0.4580	7.1470	54.20	6.0622	3	222.0	18.70	396.90	5.33	36.20

Output 11.1: Sample of the Boston house price dataset

The dataset is available in the bundle of source code provided with this book. Alternatively, you can download¹ this dataset and save it to your current working directory with the file name `housing.csv`. Reasonable performance for models evaluated using Mean Squared Error (MSE) is around 20 in thousands of dollars squared (or \$4,500 if you take the square root). This is a nice target to aim for with our neural network model.

11.2 Develop a Baseline Neural Network Model

In this section, you will create a baseline neural network model for the regression problem. Let's start by importing all the functions and objects you will need for this chapter.

```
import pandas as pd
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from scikeras.wrappers import KerasRegressor
from sklearn.model_selection import cross_val_score
from sklearn.model_selection import KFold
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
...
```

Listing 11.1: Import classes and functions

¹<https://raw.githubusercontent.com/jbrownlee/Datasets/master/housing.data>

You can now load your dataset from a file in the local directory. The dataset is, in fact, not in CSV format in the UCI Machine Learning Repository. The attributes are instead separated by whitespace. You can load this easily using the Pandas library. Then split the input (X) and output (Y) attributes, making them easier to model with Keras and scikit-learn.

```
...
# load dataset
dataframe = pd.read_csv("housing.csv", delim_whitespace=True, header=None)
dataset = dataframe.values
# split into input (X) and output (Y) variables
X = dataset[:,0:13]
Y = dataset[:,13]
```

Listing 11.2: Load dataset and separate into input and output variables

You can create Keras models and evaluate them with scikit-learn using handy wrapper objects provided by the SciKeras library. This is desirable, because scikit-learn excels at evaluating models and will allow you to use powerful data preparation and model evaluation schemes with very few lines of code. The Keras wrappers require a function as an argument. This function you must define is responsible for creating the neural network model to be evaluated.

Below, you will define the function to create the baseline model to be evaluated. It is a simple model with a single fully connected hidden layer with the same number of neurons as input attributes (13). The network uses good practices such as the rectifier activation function for the hidden layer. No activation function is used for the output layer because it is a regression problem, and you are interested in predicting numerical values directly without transformation.

The efficient ADAM optimization algorithm is used, and a mean squared error loss function is optimized. This will be the same metric you will use to evaluate the performance of the model. It is a desirable metric because taking the square root of an error value allows you to directly understand in the context of the problem with the units in thousands of dollars.

```
...
# define base model
def baseline_model():
    # create model
    model = Sequential()
    model.add(Dense(13, input_shape=(13,),
                    kernel_initializer='normal', activation='relu'))
    model.add(Dense(1, kernel_initializer='normal'))
    # Compile model
    model.compile(loss='mean_squared_error', optimizer='adam')
    return model
```

Listing 11.3: Define and compile a baseline neural network model

The SciKeras wrapper object used in scikit-learn as a regression estimator is called `KerasRegressor`. You create an instance and pass it both the name of the function to create the neural network model and some parameters to pass along to the `fit()` function of the model later, such as the number of epochs and batch size. Both of these are set to sensible defaults.

```
...
# evaluate model
estimator = KerasRegressor(model=baseline_model, epochs=100, batch_size=5, verbose=0)
```

Listing 11.4: Prepare model wrapper for scikit-learn

The final step is to evaluate this baseline model. You will use 10-fold cross-validation to evaluate the model. MSE is the metric here but since `cross_val_score()` is to find the one with highest score, `neg_mean_squared_error` (negative of MSE) is used.

```
...
kfold = KFold(n_splits=10)
results = cross_val_score(estimator, X, Y, cv=kfold, scoring='neg_mean_squared_error')
print("Results: %.2f (%.2f) MSE" % (results.mean(), results.std()))
```

Listing 11.5: Evaluate baseline model

After tying this all together, the complete example is listed below.

```
# Regression Example With Boston Dataset: Baseline
from pandas import read_csv
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from scikeras.wrappers import KerasRegressor
from sklearn.model_selection import cross_val_score
from sklearn.model_selection import KFold
# load dataset
dataframe = read_csv("housing.csv", delim_whitespace=True, header=None)
dataset = dataframe.values
# split into input (X) and output (Y) variables
X = dataset[:,0:13]
Y = dataset[:,13]
# define base model
def baseline_model():
    # create model
    model = Sequential()
    model.add(Dense(13, input_shape=(13,),
                        kernel_initializer='normal', activation='relu'))
    model.add(Dense(1, kernel_initializer='normal'))
    # Compile model
    model.compile(loss='mean_squared_error', optimizer='adam')
    return model
# evaluate model
estimator = KerasRegressor(model=baseline_model, epochs=100, batch_size=5, verbose=0)
kfold = KFold(n_splits=10)
results = cross_val_score(estimator, X, Y, cv=kfold, scoring='neg_mean_squared_error')
print("Baseline: %.2f (%.2f) MSE" % (results.mean(), results.std()))
```

Listing 11.6: Multilayer perceptron model for Boston house problem

Running this code gives you an estimate of the model's performance on the problem for unseen data. The result reports the mean squared error, including the average and standard deviation (average variance) across all 10 folds of the cross-validation evaluation.



Note: The mean squared error is negative because scikit-learn inverts so that the metric is maximized instead of minimized. You can ignore the sign of the result.



Note: Your results may vary given the stochastic nature of the algorithm or evaluation procedure, or differences in numerical precision. Consider running the example a few times and compare the average outcome.

Baseline: -32.65 (23.33) MSE

Output 11.2: Sample output from evaluating the baseline model

11.3 Modeling the Standardized Dataset

An important concern with the Boston house price dataset is that the input attributes all vary in their scales because they measure different quantities. It is almost always good practice to prepare your data before modeling it using a neural network model. Continuing from the above baseline model, you can re-evaluate the same model using a standardized version of the input dataset.

You can use scikit-learn's `Pipeline` framework to perform the standardization during the model evaluation process within each fold of the cross-validation. This ensures that there is no data leakage from each test set cross-validation fold into the training data. The code below creates a scikit-learn `Pipeline` that first standardizes the dataset and then creates and evaluates the baseline neural network model.

```
...
# evaluate model with standardized dataset
estimators = []
estimators.append(('standardize', StandardScaler()))
estimators.append(('mlp', KerasRegressor(model=baseline_model,
                                          epochs=50, batch_size=5, verbose=0)))

pipeline = Pipeline(estimators)
kfold = KFold(n_splits=10)
results = cross_val_score(pipeline, X, Y, cv=kfold, scoring='neg_mean_squared_error')
print("Standardized: %.2f (%.2f) MSE" % (results.mean(), results.std()))
```

Listing 11.7: Evaluate model with standardized dataset

After tying this together, the complete example is listed below.

```
# Regression Example With Boston Dataset: Standardized
from pandas import read_csv
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from skeras.wrappers import KerasRegressor
from sklearn.model_selection import cross_val_score
from sklearn.model_selection import KFold
```

```

from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
# load dataset
dataframe = read_csv("housing.csv", delim_whitespace=True, header=None)
dataset = dataframe.values
# split into input (X) and output (Y) variables
X = dataset[:,0:13]
Y = dataset[:,13]
# define base model
def baseline_model():
    # create model
    model = Sequential()
    model.add(Dense(13, input_shape=(13,),
                    kernel_initializer='normal', activation='relu'))
    model.add(Dense(1, kernel_initializer='normal'))
    # Compile model
    model.compile(loss='mean_squared_error', optimizer='adam')
    return model
# evaluate model with standardized dataset
estimators = []
estimators.append(('standardize', StandardScaler()))
estimators.append(('mlp', KerasRegressor(model=baseline_model,
                                         epochs=50, batch_size=5, verbose=0)))

pipeline = Pipeline(estimators)
kfold = KFold(n_splits=10)
results = cross_val_score(pipeline, X, Y, cv=kfold, scoring='neg_mean_squared_error')
print("Standardized: %.2f (%.2f) MSE" % (results.mean(), results.std()))

```

Listing 11.8: Update to use a standardized dataset

Running the example provides an improved performance over the baseline model without standardized data, dropping the error.



Note: Your results may vary given the stochastic nature of the algorithm or evaluation procedure, or differences in numerical precision. Consider running the example a few times and compare the average outcome.

```
Standardized: -29.54 (27.87) MSE
```

Output 11.3: Sample output from evaluating the model on the standardized dataset

A further extension of this section would be to similarly apply a rescaling to the output variable, such as normalizing it to the range of 0 to 1 and using a sigmoid or similar activation function on the output layer to narrow output predictions to the same range.

11.4 Tune the Neural Network Topology

Many concerns can be optimized for a neural network model. Perhaps the point of biggest leverage is the structure of the network itself, including the number of layers and the number of neurons in each layer. In this section, you will evaluate two additional network topologies

in an effort to further improve the performance of the model. You will look at both a deeper and a wider network topology.

Evaluate a Deeper Network Topology

One way to improve the performance of a neural network is to add more layers. This might allow the model to extract and recombine higher-order features embedded in the data. In this section, you will evaluate the effect of adding one more hidden layer to the model. This is as easy as defining a new function to create this deeper model, copied from your baseline model above. You can then insert a new line after the first hidden layer — in this case, with about half the number of neurons.

```
...
# define the model
def larger_model():
    # create model
    model = Sequential()
    model.add(Dense(13, input_shape=(13,),
                    kernel_initializer='normal', activation='relu'))
    model.add(Dense(6, kernel_initializer='normal', activation='relu'))
    model.add(Dense(1, kernel_initializer='normal'))
    # Compile model
    model.compile(loss='mean_squared_error', optimizer='adam')
    return model
```

Listing 11.9: Making a deeper network topology

Your network topology now looks like this:

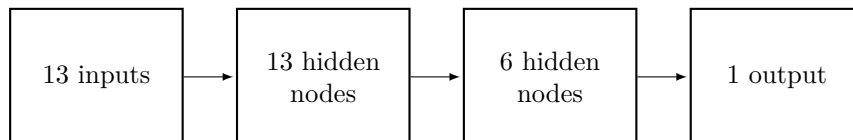


Figure 11.1: Summary of deeper network topology

You can evaluate this network topology in the same way as above, while also using the standardization of the dataset shown above to improve performance.

```
...
estimators = []
estimators.append(('standardize', StandardScaler()))
estimators.append(('mlp', KerasRegressor(model=larger_model,
                                         epochs=50, batch_size=5, verbose=0)))

pipeline = Pipeline(estimators)
kfold = KFold(n_splits=10)
results = cross_val_score(pipeline, X, Y, cv=kfold, scoring='neg_mean_squared_error')
print("Larger: %.2f (%.2f) MSE" % (results.mean(), results.std()))
```

Listing 11.10: Evaluate the house problem with standardization and deeper network

After tying this together, the complete example is listed below.

```

# Regression Example With Boston Dataset: Standardized and Larger
from pandas import read_csv
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from scikeras.wrappers import KerasRegressor
from sklearn.model_selection import cross_val_score
from sklearn.model_selection import KFold
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
# load dataset
dataframe = read_csv("housing.csv", delim_whitespace=True, header=None)
dataset = dataframe.values
# split into input (X) and output (Y) variables
X = dataset[:,0:13]
Y = dataset[:,13]
# define the model
def larger_model():
    # create model
    model = Sequential()
    model.add(Dense(13, input_shape=(13,),
                    kernel_initializer='normal', activation='relu'))
    model.add(Dense(6, kernel_initializer='normal', activation='relu'))
    model.add(Dense(1, kernel_initializer='normal'))
    # Compile model
    model.compile(loss='mean_squared_error', optimizer='adam')
    return model
# evaluate model with standardized dataset
estimators = []
estimators.append(('standardize', StandardScaler()))
estimators.append(('mlp', KerasRegressor(model=larger_model,
                                         epochs=50, batch_size=5, verbose=0)))

pipeline = Pipeline(estimators)
kfold = KFold(n_splits=10)
results = cross_val_score(pipeline, X, Y, cv=kfold, scoring='neg_mean_squared_error')
print("Larger: %.2f (%.2f) MSE" % (results.mean(), results.std()))

```

Listing 11.11: Evaluate the larger neural network model

Running this model shows a further improvement in MSE performance.



Note: Your results may vary given the stochastic nature of the algorithm or evaluation procedure, or differences in numerical precision. Consider running the example a few times and compare the average outcome.

```
Larger: -22.83 (25.33) MSE
```

Output 11.4: Sample output from evaluating the deeper model

Evaluate a Wider Network Topology

Another approach to increasing the representational capacity of the model is to create a wider network. In this section, you will evaluate the effect of keeping a shallow network architecture

and nearly doubling the number of neurons in the one hidden layer. Again, all you need to do is define a new function that creates your neural network model. Here, you will increase the number of neurons in the hidden layer compared to the baseline model from 13 to 20.

```
...
# define wider model
def wider_model():
    # create model
    model = Sequential()
    model.add(Dense(20, input_shape=(13,),
                    kernel_initializer='normal', activation='relu'))
    model.add(Dense(1, kernel_initializer='normal'))
    # Compile model
    model.compile(loss='mean_squared_error', optimizer='adam')
    return model
```

Listing 11.12: Creating a wider model

Your network topology now looks like this:

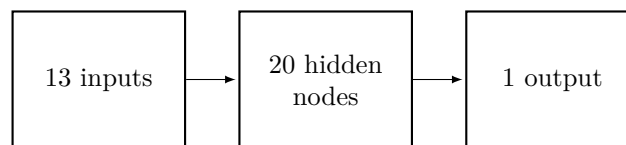


Figure 11.2: Summary of deeper network topology

You can evaluate the wider network topology using the same scheme as above:

```
...
estimators = []
estimators.append(('standardize', StandardScaler()))
estimators.append(('mlp', KerasRegressor(model=wider_model,
                                         epochs=100, batch_size=5, verbose=0)))

pipeline = Pipeline(estimators)
kfold = KFold(n_splits=10)
results = cross_val_score(pipeline, X, Y, cv=kfold, scoring='neg_mean_squared_error')
print("Wider: %.2f (%.2f) MSE" % (results.mean(), results.std()))
```

Listing 11.13: Evaluate a wider network with standardized data

After tying this together, the complete example is listed below.

```
# Regression Example With Boston Dataset: Standardized and Wider
from pandas import read_csv
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from scikeras.wrappers import KerasRegressor
from sklearn.model_selection import cross_val_score
from sklearn.model_selection import KFold
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
# load dataset
```

```

dataframe = read_csv("housing.csv", delim_whitespace=True, header=None)
dataset = dataframe.values
# split into input (X) and output (Y) variables
X = dataset[:,0:13]
Y = dataset[:,13]
# define wider model
def wider_model():
    # create model
    model = Sequential()
    model.add(Dense(20, input_shape=(13,),
                        kernel_initializer='normal', activation='relu'))
    model.add(Dense(1, kernel_initializer='normal'))
    # Compile model
    model.compile(loss='mean_squared_error', optimizer='adam')
    return model
# evaluate model with standardized dataset
estimators = []
estimators.append(('standardize', StandardScaler()))
estimators.append(('mlp', KerasRegressor(model=wider_model,
                                          epochs=100, batch_size=5, verbose=0)))

pipeline = Pipeline(estimators)
kfold = KFold(n_splits=10)
results = cross_val_score(pipeline, X, Y, cv=kfold, scoring='neg_mean_squared_error')
print("Wider: %.2f (%.2f) MSE" % (results.mean(), results.std()))

```

Listing 11.14: Evaluate the wider neural network model

Building the model reveals a further drop in error to about 21 thousand squared dollars. This is not a bad result for this problem.



Note: Your results may vary given the stochastic nature of the algorithm or evaluation procedure, or differences in numerical precision. Consider running the example a few times and compare the average outcome.

```
Wider: -21.71 (24.39) MSE
```

Output 11.5: Sample output from evaluating the wider model

It might have been hard to guess that a wider network would outperform a deeper network on this problem. The results demonstrate the importance of empirical testing in developing neural network models.

11.5 Further Readings

This section provides more resources on the topic if you are looking to go deeper.

Online Resources

Boston Housing Dataset.

<http://lib.stat.cmu.edu/datasets/boston>

SciKeras documentation.

<https://www.adriangb.com/scikeras/stable/>

KFold cross-validator. scikit-learn documentation.

https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.KFold.html

11.6 Summary

In this chapter you discovered the Keras deep learning library for modeling regression problems. Through this chapter you learned how to develop and evaluate neural network models, including:

- ▷ How to load data and develop a baseline model.
- ▷ How to lift performance using data preparation techniques like standardization.
- ▷ How to design and evaluate networks with different varying topologies on a problem.

After these examples, you should understand what Keras can do. In the next chapter, we will see several syntax variations in Keras.

Three Ways to Build Models in Keras

12

If you've looked at how other people wrote Keras models on Github, you've probably noticed that there are some different ways to create models in Keras. There's the `Sequential` model, which allows you to define an entire model in a single line, usually with some line breaks for readability. Then, there's the functional interface that allows for more complicated model architectures, and there's also the `Model` subclass which helps reusability. This chapter will explore the different ways to create models in Keras, along with their advantages and drawbacks. This will equip you with the knowledge you need to create your own machine learning models in Keras. After you complete this chapter, you will learn:

- ▷ Different ways that Keras offers to build models
- ▷ How to use the `Sequential` class, functional interface, and subclassing `keras.Model` to build Keras models
- ▷ When to use the different methods to create Keras models

Let's get started!

Overview

This chapter is in three parts, covering the different ways to build machine learning models in Keras:

- ▷ Using the `Sequential` class
- ▷ Using Keras' functional interface
- ▷ Subclassing `keras.Model`

12.1 Using the `Sequential` class

The `Sequential` model is just as the name implies. It consists of a sequence of layers, one after the other. From the Keras documentation,

“ A Sequential model is appropriate for a plain stack of layers where each layer has exactly one input tensor and one output tensor. ”
— Keras documentation

It is a simple, easy-to-use way to start building your Keras model. To start, import TensorFlow and then the `Sequential` model:

```
import tensorflow as tf
from tensorflow.keras import Sequential
```

Listing 12.1: Import statements for using *Sequential* model

Then, you can start building your machine learning model by stacking various layers together. For this example, let's build a LeNet5 model with the classic CIFAR-10 image dataset as the input:

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Input, Flatten, Conv2D, MaxPool2D

model = Sequential([
    Input(shape=(32,32,3)),
    Conv2D(6, (5,5), padding="same", activation="relu"),
    MaxPool2D(pool_size=(2,2)),
    Conv2D(16, (5,5), padding="same", activation="relu"),
    MaxPool2D(pool_size=(2, 2)),
    Conv2D(120, (5,5), padding="same", activation="relu"),
    Flatten(),
    Dense(units=84, activation="relu"),
    Dense(units=10, activation="softmax"),
])

model.summary()
```

Listing 12.2: Creating the LeNet5 network with *Sequential* model

Notice that you are just passing in an array of the layers you want your model to contain into the `Sequential` model constructor. Looking at the `model.summary()`, you can see the model's architecture.

Model: "sequential"

Layer (type)	Output Shape	Param #
=====		
conv2d (Conv2D)	(None, 32, 32, 6)	456
max_pooling2d (MaxPooling2D)	(None, 16, 16, 6)	0
conv2d_1 (Conv2D)	(None, 16, 16, 16)	2416
max_pooling2d_1 (MaxPooling2D)	(None, 8, 8, 16)	0
conv2d_2 (Conv2D)	(None, 8, 8, 120)	48120

```

flatten (Flatten)                (None, 7680)                0
dense (Dense)                    (None, 84)                  645204
dense_1 (Dense)                  (None, 10)                  850

=====
Total params: 697,046
Trainable params: 697,046
Non-trainable params: 0

```

Output 12.1: LeNet5 network

And just to test out the model, let's go ahead and load the CIFAR-10 dataset and run `model.compile()` and `model.fit()`:

```

from tensorflow as tf

(trainX, trainY), (testX, testY) = tf.keras.datasets.cifar10.load_data()
model.compile(optimizer="adam",
              loss=tf.keras.losses.SparseCategoricalCrossentropy(), metrics="acc")

history = model.fit(x=trainX, y=trainY, batch_size=256, epochs=10,
                   validation_data=(testX, testY))

```

Listing 12.3: Loading CIFAR-10 dataset and train our network

This gives us this output:

```

Epoch 1/10
196/196 [=====] - 13s 10ms/step - loss: 2.7669 - acc: 0.3648 - val_loss: 1.
Epoch 2/10
196/196 [=====] - 2s 8ms/step - loss: 1.3883 - acc: 0.5097 - val_loss: 1.36
Epoch 3/10
196/196 [=====] - 2s 8ms/step - loss: 1.2239 - acc: 0.5694 - val_loss: 1.29
Epoch 4/10
196/196 [=====] - 2s 8ms/step - loss: 1.1020 - acc: 0.6120 - val_loss: 1.26
Epoch 5/10
196/196 [=====] - 2s 8ms/step - loss: 0.9931 - acc: 0.6498 - val_loss: 1.28
Epoch 6/10
196/196 [=====] - 2s 9ms/step - loss: 0.8888 - acc: 0.6903 - val_loss: 1.31
Epoch 7/10
196/196 [=====] - 2s 8ms/step - loss: 0.7882 - acc: 0.7229 - val_loss: 1.42
Epoch 8/10
196/196 [=====] - 2s 8ms/step - loss: 0.6915 - acc: 0.7582 - val_loss: 1.45
Epoch 9/10
196/196 [=====] - 2s 8ms/step - loss: 0.5934 - acc: 0.7931 - val_loss: 1.53
Epoch 10/10
196/196 [=====] - 2s 8ms/step - loss: 0.5113 - acc: 0.8214 - val_loss: 1.63

```

Output 12.2: Progress during the training of our network

That's pretty good for a first pass at a model. Putting the code for LeNet5 using a `Sequential` model together, you have:

```
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense, Input, Flatten, Conv2D, MaxPool2D

(trainX, trainY), (testX, testY) = tf.keras.datasets.cifar10.load_data()

model = Sequential([
    Input(shape=(32,32,3)),
    Conv2D(6, (5,5), padding="same", activation="relu"),
    MaxPool2D(pool_size=(2,2)),
    Conv2D(16, (5,5), padding="same", activation="relu"),
    MaxPool2D(pool_size=(2, 2)),
    Conv2D(120, (5,5), padding="same", activation="relu"),
    Flatten(),
    Dense(units=84, activation="relu"),
    Dense(units=10, activation="softmax"),
])

model.summary()

model.compile(optimizer="adam",
              loss=tf.keras.losses.SparseCategoricalCrossentropy(), metrics="acc")
history = model.fit(x=trainX, y=trainY, batch_size=256, epochs=10,
                   validation_data=(testX, testY))
```

Listing 12.4: Complete example of building a network with *Sequential* and training

Now, let's explore what the other ways of constructing Keras models can do, starting with the functional interface!

12.2 Using Keras' Functional Interface

The next method of constructing Keras models you will explore uses Keras' functional interface. The functional interface uses the layers as functions instead, taking in a Tensor and outputting a Tensor as well. The functional interface is a more flexible way of representing a Keras model as you are not restricted only to sequential models with layers stacked on top of one another. Instead, you can build models that branch into multiple paths, have multiple inputs, etc.

Consider an Add layer that takes inputs from two or more paths and adds the tensors together.

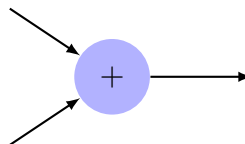


Figure 12.1: Add layer with two inputs

Since this cannot be represented as a linear stack of layers due to the multiple inputs, you are unable to define it using a *Sequential* object. Here's where Keras' functional interface comes in. You can define an Add layer with two input tensors as such:

```
from tensorflow.keras.layers import Add
add_layer = Add()([layer1, layer2])
```

Listing 12.5: Using *Add* layer to combine two other layers

Now that you've seen a quick example of the functional interface, let's take a look at what the LeNet5 model that you defined by instantiating a `Sequential` class would look like using a functional interface.

```
import tensorflow as tf
from tensorflow.keras.layers import Dense, Input, Flatten, Conv2D, MaxPool2D
from tensorflow.keras.models import Model

input_layer = Input(shape=(32,32,3))
x = Conv2D(6, (5,5), padding="same", activation="relu")(input_layer)
x = MaxPool2D(pool_size=(2,2))(x)
x = Conv2D(16, (5,5), padding="same", activation="relu")(x)
x = MaxPool2D(pool_size=(2, 2))(x)
x = Conv2D(120, (5,5), padding="same", activation="relu")(x)
x = Flatten()(x)
x = Dense(units=84, activation="relu")(x)
x = Dense(units=10, activation="softmax")(x)

model = Model(inputs=input_layer, outputs=x)

model.summary()
```

Listing 12.6: Creating a LeNet5 network with functional interface

And looking at the model summary:

Model: "model"

Layer (type)	Output Shape	Param #
=====		
input_1 (InputLayer)	[(None, 32, 32, 3)]	0
conv2d (Conv2D)	(None, 32, 32, 6)	456
max_pooling2d (MaxPooling2D)	(None, 16, 16, 6)	0
conv2d_1 (Conv2D)	(None, 16, 16, 16)	2416
max_pooling2d_1 (MaxPooling2D)	(None, 8, 8, 16)	0
conv2d_2 (Conv2D)	(None, 8, 8, 120)	48120
flatten (Flatten)	(None, 7680)	0
dense (Dense)	(None, 84)	645204
dense_1 (Dense)	(None, 10)	850
=====		
Total params: 697,046		


```
Trainable params: 697,046
Non-trainable params: 0
```

Output 12.3: LeNet5 model as creating by functional interface

As you can see, the model architecture is the same for both LeNet5 models you implemented using the functional interface or the `Sequential` class.

Now that you've seen how to use Keras' functional interface, let's look at a model architecture that you can implement using the functional interface but not with the `Sequential` class. For this example, look at the residual block introduced in ResNet. Visually, the residual block looks like this:

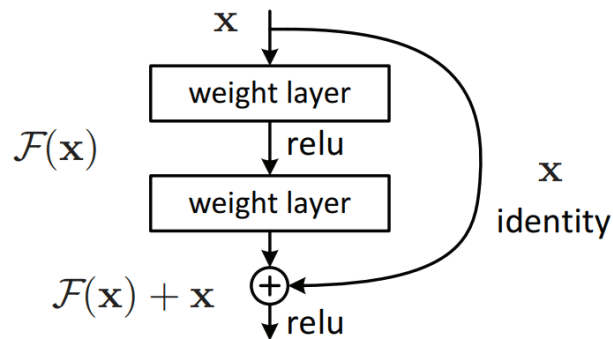


Figure 12.2: Residual block as used in ResNet

You can see that a model defined using the `Sequential` class would be unable to construct such a block due to the skip connection, which prevents this block from being represented as a simple stack of layers. Using the functional interface this is one way you can define a ResNet block:

```
def residual_block(x, filters):
    # store the input tensor to be added later as the identity
    identity = x
    x = Conv2D(filters, (3, 3), strides = (1, 1), padding="same")(x)
    x = BatchNormalization()(x)
    x = relu(x)
    x = Conv2D(filters, (3, 3), padding="same")(x)
    x = BatchNormalization()(x)
    x = Add()([identity, x])
    x = relu(x)

    return x
```

Listing 12.7: Residual block in Keras

Then, you can build a simple network using these residual blocks using the functional interface:

```
input_layer = Input(shape=(32,32,3))
x = Conv2D(32, (3, 3), padding="same", activation="relu")(input_layer)
x = residual_block(x, 32)
x = Conv2D(64, (3, 3), strides=(2, 2), padding="same",
```

```

        activation="relu")(x)
x = residual_block(x, 64)
x = Conv2D(128, (3, 3), strides=(2, 2), padding="same",
        activation="relu")(x)
x = residual_block(x, 128)
x = Flatten()(x)
x = Dense(units=84, activation="relu")(x)
x = Dense(units=10, activation="softmax")(x)

model = Model(inputs=input_layer, outputs = x)
model.summary()

model.compile(optimizer="adam", loss=tf.keras.losses.SparseCategoricalCrossentropy(),
        metrics="acc")

history = model.fit(x=trainX, y=trainY, batch_size=256, epochs=10,
        validation_data=(testX, testY))

```

Listing 12.8: A neural network with residual block

Running this code and looking at the model summary and training results:

Model: "model"

Layer (type)	Output Shape	Param #	Connected to
input_1 (InputLayer)	[(None, 32, 32, 3)]	0	[]
conv2d (Conv2D)	(None, 32, 32, 32)	896	['input_1[0][0]']
conv2d_1 (Conv2D)	(None, 32, 32, 32)	9248	['conv2d[0][0]']
batch_normalization (BatchNormalization)	(None, 32, 32, 32)	128	['conv2d_1[0][0]']
tf.nn.relu (TFOpLambda)	(None, 32, 32, 32)	0	['batch_normalization[0][0]']
conv2d_2 (Conv2D)	(None, 32, 32, 32)	9248	['tf.nn.relu[0][0]']
batch_normalization_1 (BatchNormalization)	(None, 32, 32, 32)	128	['conv2d_2[0][0]']
add (Add)	(None, 32, 32, 32)	0	['conv2d[0][0]', 'batch_normalization_1[0][0]']
tf.nn.relu_1 (TFOpLambda)	(None, 32, 32, 32)	0	['add[0][0]']
conv2d_3 (Conv2D)	(None, 16, 16, 64)	18496	['tf.nn.relu_1[0][0]']
conv2d_4 (Conv2D)	(None, 16, 16, 64)	36928	['conv2d_3[0][0]']
batch_normalization_2 (BatchNormalization)	(None, 16, 16, 64)	256	['conv2d_4[0][0]']
tf.nn.relu_2 (TFOpLambda)	(None, 16, 16, 64)	0	['batch_normalization_2[0][0]']

conv2d_5 (Conv2D)	(None, 16, 16, 64)	36928	['tf.nn.relu_2[0][0]']
batch_normalization_3 (Batch Normalization)	(None, 16, 16, 64)	256	['conv2d_5[0][0]']
add_1 (Add)	(None, 16, 16, 64)	0	['conv2d_3[0][0]', 'batch_normalization_3[0][0]']
tf.nn.relu_3 (TFOpLambda)	(None, 16, 16, 64)	0	['add_1[0][0]']
conv2d_6 (Conv2D)	(None, 8, 8, 128)	73856	['tf.nn.relu_3[0][0]']
conv2d_7 (Conv2D)	(None, 8, 8, 128)	147584	['conv2d_6[0][0]']
batch_normalization_4 (Batch Normalization)	(None, 8, 8, 128)	512	['conv2d_7[0][0]']
tf.nn.relu_4 (TFOpLambda)	(None, 8, 8, 128)	0	['batch_normalization_4[0][0]']
conv2d_8 (Conv2D)	(None, 8, 8, 128)	147584	['tf.nn.relu_4[0][0]']
batch_normalization_5 (Batch Normalization)	(None, 8, 8, 128)	512	['conv2d_8[0][0]']
add_2 (Add)	(None, 8, 8, 128)	0	['conv2d_6[0][0]', 'batch_normalization_5[0][0]']
tf.nn.relu_5 (TFOpLambda)	(None, 8, 8, 128)	0	['add_2[0][0]']
flatten (Flatten)	(None, 8192)	0	['tf.nn.relu_5[0][0]']
dense (Dense)	(None, 84)	688212	['flatten[0][0]']
dense_1 (Dense)	(None, 10)	850	['dense[0][0]']

```

=====
Total params: 1,171,622
Trainable params: 1,170,726
Non-trainable params: 896

```

```

Epoch 1/10
196/196 [=====] - 21s 46ms/step - loss: 3.4463 - acc: 0.3635 - val_loss: 1.
Epoch 2/10
196/196 [=====] - 8s 43ms/step - loss: 1.3267 - acc: 0.5200 - val_loss: 1.3
Epoch 3/10
196/196 [=====] - 8s 43ms/step - loss: 1.1095 - acc: 0.6062 - val_loss: 1.2
Epoch 4/10
196/196 [=====] - 9s 44ms/step - loss: 0.9618 - acc: 0.6585 - val_loss: 1.5
Epoch 5/10
196/196 [=====] - 9s 44ms/step - loss: 0.8656 - acc: 0.6968 - val_loss: 1.1
Epoch 6/10
196/196 [=====] - 8s 43ms/step - loss: 0.7622 - acc: 0.7361 - val_loss: 1.1
Epoch 7/10
196/196 [=====] - 9s 44ms/step - loss: 0.6801 - acc: 0.7602 - val_loss: 1.1
Epoch 8/10

```

```

196/196 [=====] - 8s 43ms/step - loss: 0.6106 - acc: 0.7905 - val_loss: 1.1
Epoch 9/10
196/196 [=====] - 9s 43ms/step - loss: 0.5367 - acc: 0.8146 - val_loss: 1.2
Epoch 10/10
196/196 [=====] - 9s 47ms/step - loss: 0.4776 - acc: 0.8348 - val_loss: 1.0

```

Output 12.4: A neural network with residual block

And combining the code for our simple network using residual blocks:

```

import tensorflow as tf
from tensorflow.keras.layers import Input, Conv2D, BatchNormalization, Add, \
    MaxPool2D, Flatten, Dense
from tensorflow.keras.activations import relu
from tensorflow.keras.models import Model

def residual_block(x, filters):
    # store the input tensor to be added later as the identity
    identity = x
    x = Conv2D(filters = filters, kernel_size=(3, 3), strides = (1, 1), padding="same")(x)
    x = BatchNormalization()(x)
    x = relu(x)
    x = Conv2D(filters = filters, kernel_size=(3, 3), padding="same")(x)
    x = BatchNormalization()(x)
    x = Add()([identity, x])
    x = relu(x)

    return x

(trainX, trainY), (testX, testY) = tf.keras.datasets.cifar10.load_data()

input_layer = Input(shape=(32,32,3))
x = Conv2D(32, (3, 3), padding="same", activation="relu")(input_layer)
x = residual_block(x, 32)
x = Conv2D(64, (3, 3), strides=(2, 2), padding="same", activation="relu")(x)
x = residual_block(x, 64)
x = Conv2D(128, (3, 3), strides=(2, 2), padding="same", activation="relu")(x)
x = residual_block(x, 128)
x = Flatten()(x)
x = Dense(units=84, activation="relu")(x)
x = Dense(units=10, activation="softmax")(x)

model = Model(inputs=input_layer, outputs = x)
model.summary()

model.compile(optimizer="adam", loss=tf.keras.losses.SparseCategoricalCrossentropy(),
    metrics="acc")

history = model.fit(x=trainX, y=trainY, batch_size=256, epochs=10,
    validation_data=(testX, testY))

```

Listing 12.9: Complete code to build a neural network with residual block using functional interface

12.3 Subclassing `keras.Model`

Keras also provides an object-oriented approach to creating models, which helps with reusability and allows you to represent the models you want to create as classes. This representation might be more intuitive since you can think about models as a set of layers strung together to form your network.

To begin subclassing `keras.Model`, you first need to import it:

```
from tensorflow.keras.models import Model
```

Listing 12.10: Import statement for subclassing

Then, you can start subclassing `Model`. First, you need to build the layers that you want to use in your method calls since you only want to instantiate these layers once instead of each time you call your model. To keep in line with previous examples, let's build a LeNet5 model here as well.

```
class LeNet5(tf.keras.Model):
    def __init__(self):
        super(LeNet5, self).__init__()
        #creating layers in initializer
        self.conv1 = Conv2D(6, (5,5), padding="same", activation="relu")
        self.max_pool2x2 = MaxPool2D(pool_size=(2,2))
        self.conv2 = Conv2D(16, (5,5), padding="same", activation="relu")
        self.conv3 = Conv2D(120, (5,5), padding="same", activation="relu")
        self.flatten = Flatten()
        self.fc2 = Dense(units=84, activation="relu")
        self.fc3 = Dense(units=10, activation="softmax")
```

Listing 12.11: Building a network using subclassing

Then, override the `call` method to define what happens when the model is called. You override it with your model, which uses the layers that you have built in the initializer.

```
def call(self, input_tensor):
    # don't create layers here, need to create the layers in initializer,
    # otherwise you will get the tf.Variable can only be created once error
    conv1 = self.conv1(input_tensor)
    maxpool1 = self.max_pool2x2(conv1)
    conv2 = self.conv2(maxpool1)
    maxpool2 = self.max_pool2x2(conv2)
    conv3 = self.conv3(maxpool2)
    flatten = self.flatten(conv3)
    fc2 = self.fc2(flatten)
    fc3 = self.fc3(fc2)

    return fc3
```

Listing 12.12: The `call()` method added to the subclassed model

It is important to have all the layers created at the class constructor, not inside the `call()` method. This is because the `call()` method will be invoked multiple times with different input

tensors. But you want to use the same layer objects in each call to optimize their weight. You can then instantiate your new LeNet5 class and use it as part of a model:

```
input_layer = Input(shape=(32,32,3,))
x = LeNet5()(input_layer)

model = Model(inputs=input_layer, outputs=x)

model.summary(expand_nested=True)
```

Listing 12.13: Creating a LeNet5 network using subclassing

And you can see that the model has the same number of parameters as the previous two versions of LeNet5 that were built previously and has the same structure within it:

```
Model: "model"
```

Layer (type)	Output Shape	Param #
input_1 (InputLayer)	[(None, 32, 32, 3)]	0
le_net5 (LeNet5)	(None, 10)	697046
conv2d (Conv2D)	multiple	456
max_pooling2d (MaxPooling2D)	multiple	0
conv2d_1 (Conv2D)	multiple	2416
conv2d_2 (Conv2D)	multiple	48120
flatten (Flatten)	multiple	0
dense (Dense)	multiple	645204
dense_1 (Dense)	multiple	850

```

Total params: 697,046
Trainable params: 697,046
Non-trainable params: 0
```

Output 12.5: LeNet5 network as created with subclassing

Combining all the code to create your LeNet5 subclass of `keras.Model`:

```
import tensorflow as tf
from tensorflow.keras.layers import Dense, Input, Flatten, Conv2D, MaxPool2D
from tensorflow.keras.models import Model

class LeNet5(tf.keras.Model):
    def __init__(self):
```

```

super(LeNet5, self).__init__()
#creating layers in initializer
self.conv1 = Conv2D(6, (5,5), padding="same", activation="relu")
self.max_pool2x2 = MaxPool2D(pool_size=(2,2))
self.conv2 = Conv2D(16, (5,5), padding="same", activation="relu")
self.conv3 = Conv2D(120, (5,5), padding="same", activation="relu")
self.flatten = Flatten()
self.fc2 = Dense(units=84, activation="relu")
self.fc3=Dense(units=10, activation="softmax")

def call(self, input_tensor):
    # don't add layers here, need to create the layers in initializer,
    # otherwise you will get the tf.Variable can only be created once error
    x = self.conv1(input_tensor)
    x = self.max_pool2x2(x)
    x = self.conv2(x)
    x = self.max_pool2x2(x)
    x = self.conv3(x)
    x = self.flatten(x)
    x = self.fc2(x)
    x = self.fc3(x)
    return x

input_layer = Input(shape=(32,32,3,))
x = LeNet5()(input_layer)
model = Model(inputs=input_layer, outputs=x)
model.summary(expand_nested=True)

```

Listing 12.14: Example of creating a neural network using subclassing

12.4 Further Reading

This section provides more resources on the topic if you are looking to go deeper.

Online Resources

The Sequential class. Keras API Reference.

<https://keras.io/api/models/sequential/>

The Functional API. Keras Developer Guides.

https://keras.io/guides/functional_api/

Making new layers and models via subclassing. Keras Developer Guides.

https://keras.io/guides/making_new_layers_and_models_via_subclassing/

Papers

Kaiming He et al. “Deep Residual Learning for Image Recognition”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 2016, pp. 770–778.

<https://arxiv.org/pdf/1512.03385.pdf>

(The ResNet paper).

12.5 Summary

In this chapter, you have seen three different ways to create models in Keras. In particular, this includes using the `Sequential` class, functional interface, and subclassing `keras.Model`. You have also seen examples of the same LeNet5 model being built using the different methods and a use case that can be done using the functional interface but not with the `Sequential` class.

Specifically, you learned:

- ▷ Different ways that Keras offers to build models
- ▷ How to use the `Sequential` class, functional interface, and subclassing `keras.Model` to build Keras models
- ▷ When to use the different methods to create Keras models

In the next part, you will see more details on how to fine tune a Keras model.